



BRNO UNIVERSITY OF TECHNOLOGY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF INTELLIGENT SYSTEMS

ÚSTAV INTELIGENTNÍCH SYSTÉMŮ

AN ENVIRONMENT FOR THE VISUAL DESIGN, MONITORING, AND MANAGEMENT OF DISTRIBUTED CONTROL SYSTEMS AND IOT NETWORKS

PROSTŘEDÍ PRO VIZUÁLNÍ NÁVRH, MONITOROVÁNÍ A SPRÁVU DISTRIBUOVANÝCH ŘÍDICÍCH SYSTÉMŮ A IOT SÍTÍ

BACHELOR'S THESIS

BAKALÁŘSKÁ PRÁCE

AUTHOR

AUTOR PRÁCE

JAKUB KAPITULCIN

SUPERVISOR

VEDOUČÍ PRÁCE

doc. Ing. VLADIMÍR JANOUŠEK, Ph.D.

BRNO 2026

Abstract

This thesis presents Aetherium Automata, a platform for the visual design, deployment, and monitoring of distributed IoT control systems modeled as Extended Finite State Machines (EFSMs). The platform consists of a portable C++17 runtime engine, a real-time Elixir/Phoenix gateway, and an Electron-based visual IDE. The EFSM model supports five transition types—classic guarded, timed, event-driven, probabilistic, and immediate—expressed in a YAML-based domain-specific language. The engine executes Lua scripts for guards and state actions and targets heterogeneous hardware including desktop systems, ESP32, and Raspberry Pi Pico. The IDE provides live state monitoring, variable inspection, and time-travel debugging via execution trace replay. Structural analysis using Petri net formalism enables deadlock detection and bottleneck identification across multi-automata networks. The system is demonstrated through a catalog of 18 showcase automata covering a range of real-world IoT scenarios.

Abstrakt

Tato práce představuje platformu Aetherium Automata pro vizuální návrh, nasazení a monitorování distribuovaných řídicích systémů IoT modelovaných jako rozšířené konečné automaty. Platforma se skládá z přenositelného runtime jádra v jazyce C++17, brány v Elixiru a Phoenix, a vizuálního vývojového prostředí postaveného na technologii Electron. Model automatu podporuje pět typů přechodů a doménově specifický jazyk v YAML. Jádro je spustitelné na stolních počítačích i na vestavěných platformách jako ESP32 a Raspberry Pi Pico. IDE umožňuje živé monitorování, inspekci proměnných a ladění s přetáčením času. Strukturální analýza pomocí Petriho sítí umožňuje detekci deadlocků a identifikaci úzkých hrdel v sítích automatů.

Keywords

extended finite state machine, Petri net, IoT, distributed control systems, state machine, Lua, embedded systems, visual IDE, fault injection, time-travel debugging

Klíčová slova

rozšířený konečný automat, Petriho síť, IoT, distribuované řídicí systémy, stavový stroj, Lua, vestavěné systémy, vizuální IDE, vstřikování chyb, ladění s přetáčením času

Reference

KAPITULCIN, Jakub. *An environment for the visual design, monitoring, and management of distributed control systems and IoT networks*. Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor doc. Ing. Vladimír Janoušek, Ph.D.

An environment for the visual design, monitoring, and management of distributed control systems and IoT networks

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of doc. Ing. Vladimír Janoušek, Ph.D. I have listed all the literary sources, publications, and other sources which were used during the preparation of this thesis. I used Claude Code (an AI programming assistant by Anthropic) for code assistance and documentation generation during the development of the implemented system.

.....
Jakub Kapitulcin
May 12, 2026

Acknowledgements

I would like to thank doc. Ing. Vladimír Janoušek, Ph.D. for his guidance throughout the work on this thesis.

Contents

1	Introduction	5
1.1	Design Philosophy	6
1.2	Goals	7
1.3	Thesis Structure	7
2	Background	9
2.1	Finite State Machines	9
2.2	Extended Finite State Machines	9
2.3	Petri Nets	10
2.4	Record-and-Replay Debugging	11
3	Analysis of Existing Solutions	12
3.1	Node-RED	12
3.2	Eclipse 4diac	13
3.3	Comparison	15
3.4	Requirements	16
4	System Architecture	18
4.1	Technology Selection Rationale	18
4.2	Component Responsibilities	20
4.3	Component Interaction and Data Flow	21
4.4	Deployment Model	22
5	Automata Model and DSL	23
5.1	Model Overview	23
5.2	States	23
5.3	Variables	23
5.4	Transition Types	24
5.5	Transition Resolution Algorithm	25
5.6	Black-Box Contract	26
5.7	YAML Domain-Specific Language	27
6	Runtime Engine	29
6.1	Execution Tick Cycle	29
6.2	Platform Abstraction	30
6.3	Lua Scripting	31
6.4	Timer Management	32
6.5	Fault Injection	32

6.6	Execution Trace and Time-Travel Debugging	34
7	Gateway and Communication Protocol	36
7.1	Binary Wire Protocol	36
7.2	Phoenix Gateway	37
7.3	End-to-End Message Flow	38
8	Integrated Development Environment	39
8.1	Architecture	39
8.2	Phoenix Gateway Service	39
8.3	Panels	39
9	Petri Net Analysis	41
9.1	Formal Definitions	41
9.2	Lifting a Single Automaton	41
9.3	Network Composition	43
9.4	Analysis Operations	43
9.5	Showcase Scenarios	44
10	Showcase Catalog	45
10.1	Selected Showcase: Quality Router (03)	45
10.2	Selected Showcase: Sensor Watchdog Recovery (04)	46
11	End-to-End Demonstration	48
11.1	Infrastructure Setup	48
11.2	Opening the Demo Project	49
11.3	Deploying Automata	49
11.4	Live Runtime Monitoring	50
11.5	Injecting Faults	51
11.6	Time-Travel Debugging	52
11.7	Petri Net Analysis	53
11.8	Analyzer Panel	54
11.9	Black-Box Devices	55
11.10	Demonstration Summary	56
12	Conclusion	57
12.1	Summary	57
12.2	Future Work	58
	Bibliography	59
A	Contents of the Included Storage Media	62
B	YAML DSL Schema Reference	63
C	Supported Platform Targets	64

List of Figures

4.1	High-level component architecture of Aetherium Automata.	18
5.1	Black-box contract boundary. The visible contract layer exposes named input/output ports, observable states, and emitted events to the external environment. The internal implementation — states, variables, and transitions — is opaque.	26
6.1	Engine execution tick cycle. A transition fires at most once per tick per automaton.	30
6.2	Platform abstraction layer. The runtime core depends only on the three interfaces; concrete implementations are selected at link time.	31
6.3	FaultProfile message pipeline. Each frame passes through drop, duplicate, and delay decisions independently for ingress (server → device) and egress (device → server). All probabilistic decisions draw from a seeded <code>IRandomSource</code> , making fault scenarios fully reproducible.	33
6.4	Time-travel rewind sequence. The IDE dispatches <code>rewind_deployment</code> to the gateway; the server reconstructs the automaton state at timestamp <i>T</i> via <code>replay_state_at</code> and delivers a <code>RestoreState(0x48)</code> binary command to the device, which pauses, applies the state snapshot atomically, and resumes.	35
7.1	Binary message envelope. The 2-byte platform tag <code>0xAE01</code> identifies the Aetherium Automata protocol family (v1); the following protocol version byte allows the same platform to evolve its wire format independently.	36
7.2	End-to-end message sequence for deploy, start, and runtime I/O exchange.	38
9.1	Two automata (A and B) lifted to Petri net subnets. The shared channel place <code>ch_signal</code> connects the output transition of automaton A to an input transition of automaton B.	42
10.1	Sensor watchdog automaton. Dashed transitions are timed; solid transitions are classic guards. The <code>failed</code> state is a terminal escalation sink.	46
11.1	Network panel showing the demo project with one connected device.	49
11.2	Runtime Monitor panel after deploying the sensor watchdog automaton to <code>device_cpp_01</code>	50
11.3	Runtime Monitor panel displaying live state and variable values for the deployed automaton.	51
11.4	Gateway panel with an active delay fault profile on the demo device.	52
11.5	Execution trace replay timeline in the Runtime Monitor panel.	53
11.6	Petri Net panel showing token flow for the power-contention scenario.	54

11.7 Analyzer panel reporting channel activity and an anomaly for the signal chain scenario.	55
---	----

Chapter 1

Introduction

The Internet of Things (IoT) connects increasingly large numbers of heterogeneous devices that must cooperate to execute control logic in real time. Such systems are inherently distributed and stateful: a smart thermostat cycles through heating, idle, and cooling modes; a production-line robot arm progresses through pick, move, and place phases; a sensor watchdog transitions between monitoring, alarming, and recovery states. The heterogeneity of devices, communication protocols, and data formats in large IoT deployments is a well-documented integration challenge [15]. Despite the pervasiveness of state-driven behavior, most existing IoT toolchains treat control logic as ad-hoc scripts embedded in event callbacks or workflow DAGs, making the underlying state structure implicit and difficult to reason about.

The consequence is a practical problem: when a system deployed across dozens of devices misbehaves, developers must reconstruct the global state from scattered log messages, often without any ability to replay or visually inspect the sequence of events that led to the fault. Testing distributed IoT systems reliably requires simulating faults and reproducing timing-sensitive failure scenarios [2], a capability that most existing IoT platforms do not provide natively.

This thesis addresses that problem by presenting *Aetherium Automata*, a complete toolchain for the visual design, deployment, and monitoring of distributed IoT control systems modeled as *Extended Finite State Machines* (EFSMs). The system treats automata as first-class citizens: every device behavior is expressed as a named state machine with typed variables, Lua-scripted guards and actions, and a rich set of transition types. The toolchain includes:

- a portable C++17 runtime engine that executes EFSM definitions on targets ranging from desktop Linux to ESP32 and Raspberry Pi Pico,
- a YAML-based domain-specific language (DSL) for authoring automata,
- a real-time Elixir/Phoenix gateway that bridges IDE and device network,
- an Electron-based visual IDE for design, live monitoring, fault injection, and time-travel debugging,
- structural analysis using Petri net formalism for deadlock and bottleneck detection across multi-automata networks.

1.1 Design Philosophy

Two overarching principles shaped every design decision in Aetherium Automata, from the choice of transition types to the structure of the Petri net analysis layer. They are stated here because understanding them makes the architectural choices in later chapters self-evident rather than arbitrary.

Reality-in-the-Loop. Distributed IoT systems exhibit failure modes that are invisible to unit tests: timing races between devices, cascading state-machine anomalies triggered by intermittent connections, recovery paths that are never exercised under nominal conditions. Traditional testing approaches are insufficient because the bugs live in the *interactions* between components, not inside any single unit.

Reality-in-the-Loop (RitL) is the principle that testing and production share the same execution substrate. The production automaton runs on the real (or emulated) hardware; a fault injection layer perturbs the communication boundary at configurable probability. Three features of Aetherium together close the loop:

1. **Probabilistic and stochastic transitions.** Rare real-world events — sensor dropout, power glitch, packet burst — are modelled as transitions with configurable probability weights. A transition that fires 0.1% of the time in production can be promoted to 100% for a stress run without touching the automaton definition. Events that may never occur in a real deployment can be reliably induced in the lab.
2. **Black-box contracts.** The interface under test is formally stated: input ports, output ports, observable states, emitted events. The fault injector operates at this boundary, not inside the implementation. This means external devices and third-party subsystems can be subjected to the same testing discipline as first-party automata.
3. **Time-travel debugging.** When an injected fault reveals a misbehaviour, the developer does not need to reproduce it from scratch. The execution trace is rewound to the exact state preceding the anomaly; from there, forward steps can be taken under full variable inspection. The fault is observed, not just logged.

Together these features form a test-debug cycle that is not possible in toolchains where the state machine is implicit and the communication layer is opaque.

Deployment-Aware Development. In most IoT toolchains, the developer writes logic and deploys it into a communication layer they do not own. Optimisation is retrospective: instrument the deployment, collect logs, correlate events across devices. The developer must infer what the network is doing.

Deployment-Aware Development (DAD) inverts this relationship. Because Aetherium owns the communication layer end-to-end — binary wire protocol, custom gateway, structured telemetry — every deployed automaton is accompanied by live metadata: round-trip time, connection type, battery level, wake-up timer intervals. This information is not scraped from logs; it appears in the same IDE panel as the running state machine. The developer can therefore make quantified, deliberate trade-offs:

- A measured RTT of 350 ms informs the choice of timer-transition intervals.

- A battery reading of 12 % can trigger an immediate transition to a low-power mode before the device disconnects.
- A known-intermittent connection can be reproduced via the fault injector to verify recovery behavior before it occurs in production.

The Petri net analysis layer is the highest expression of this principle. Lifting individual automata to a Petri net and composing them produces a single structural model of the entire network. Where the device view shows each automaton in isolation, the Petri net view reveals the properties that emerge from their interaction: where tokens (resources, messages, control signals) accumulate into bottlenecks, which state combinations lead to deadlock, which transitions compete for shared resources. This is analogous to acquiring a third spatial dimension: structural properties that are invisible when inspecting automata one at a time become immediately evident when the full network is projected onto a Petri net.

1.2 Goals

The primary goals of this thesis are:

1. Design and implement an EFSM model expressive enough to cover the control logic of real-world IoT devices, including time-driven, event-driven, and probabilistic behaviour — the latter being essential for Reality-in-the-Loop testing.
2. Implement a portable C++ runtime engine capable of executing the model on both desktop and constrained embedded platforms.
3. Design a YAML domain-specific language for authoring automata definitions.
4. Implement a real-time gateway and IDE that allow an operator to deploy, monitor, and debug automata across a network of heterogeneous devices, with full deployment metadata available to the developer.
5. Provide a fault injection subsystem and execution trace replay (time-travel debugging) to support Reality-in-the-Loop testing.
6. Provide a Petri net analysis layer that lifts the multi-automata network to a higher structural plane, enabling Deployment-Aware reasoning about the system as a whole.
7. Validate the system with a catalog of representative IoT showcase automata.

1.3 Thesis Structure

Section 1.1 above introduces the two guiding principles (Reality-in-the-Loop and Deployment-Aware Development) that inform all subsequent design decisions. Chapter 2 establishes the theoretical background: Finite State Machines, Extended Finite State Machines, Petri nets, and Record-and-Replay Debugging. Chapter 3 analyses existing tools (Node-RED, Eclipse 4diac), compares programmer mental models, and derives requirements. Chapter 4 describes the overall system architecture. Chapter 5 specifies the automata model and the YAML DSL. Chapter 6 covers the C++ runtime engine, including fault injection and execution trace recording. Chapter 7 describes the gateway and binary wire protocol. Chapter 8

covers the IDE panels. Chapter 9 describes the Petri net analysis layer. Chapter 10 presents the showcase catalog. Chapter 11 walks through an end-to-end demonstration. Chapter 12 concludes with results and future work.

Chapter 2

Background

2.1 Finite State Machines

A *Finite State Machine* (FSM) is a well-established mathematical formalism for modeling systems that can be in exactly one of a finite number of states at any given time and that change state in response to inputs [8]. Formally, a deterministic FSM is defined as a 5-tuple:

$$M = (Q, \Sigma, \delta, q_0, F) \quad (2.1)$$

where:

- Q is a finite, non-empty set of states,
- Σ is a finite input alphabet (set of events or signals),
- $\delta : Q \times \Sigma \rightarrow Q$ is the transition function,
- $q_0 \in Q$ is the initial state,
- $F \subseteq Q$ is the set of accepting (final) states.

In control applications the acceptance criterion is typically not used; the focus lies instead on the transition function and any actions associated with states or transitions. Two classical variants are distinguished:

Moore machine Output depends only on the current state. Output is produced upon entering a state.

Mealy machine Output depends on both the current state and the current input. Output is produced when a transition fires.

Aetherium Automata supports both semantics simultaneously through per-state code hooks executed on entry and per-tick, and per-transition code executed when a transition fires.

2.2 Extended Finite State Machines

A plain FSM is insufficient for most practical control problems because all variation must be encoded into distinct states. An *Extended Finite State Machine* (EFSM) augments

the FSM with a variable store, transforming guards and actions into functions over that store [7]:

$$M_E = (Q, \Sigma, V, \delta_E, \lambda, q_0) \quad (2.2)$$

where:

- Q, Σ, q_0 are as in the standard FSM,
- V is a finite set of typed variables (the *data context*),
- $\delta_E : Q \times \Sigma \times \text{Bool}(V) \rightarrow Q$ is the extended transition function; each transition edge carries a *guard* $g : V \rightarrow \text{Bool}$ that must be satisfied for the transition to fire,
- $\lambda : Q \times \Sigma \times V \rightarrow V$ is the output/action function that updates the variable store when a transition fires.

In Aetherium, the variable store V contains three classes of variables: *input* variables (written by the external environment, read-only to the automaton), *output* variables (written by the automaton, consumed externally), and *internal* variables (read-write by the automaton alone). Guards and actions are expressed as Lua [10, 9] scripts evaluated against V at runtime. Lua was specifically designed as a lightweight, embeddable extension language [10], making it well suited for use inside a portable C++ runtime with constrained memory budgets.

Each state $q \in Q$ additionally carries three optional code hooks evaluated in the following order:

1. **on_enter** — executed once when the state is entered,
2. **body** — executed on every execution tick while the machine remains in the state,
3. **on_exit** — executed once immediately before leaving the state.

This structure corresponds to a Moore/Mealy hybrid: **on_enter** provides state-dependent output (Moore), while the transition **body** hook provides input-dependent output (Mealy).

2.3 Petri Nets

A *Petri net* is a mathematical formalism designed for modeling concurrent, distributed, and asynchronous systems [18]. It generalizes finite automata by allowing multiple simultaneous active conditions (tokens in multiple places). A formal survey with analysis properties is given by Murata [14]. A Petri net is defined as a 4-tuple:

$$N = (P, T, F, M_0) \quad (2.3)$$

where:

- P is a finite set of *places* (representing conditions or states),
- T is a finite set of *transitions*, with $P \cap T = \emptyset$,
- $F \subseteq (P \times T) \cup (T \times P)$ is the *flow relation* (directed arcs),
- $M_0 : P \rightarrow \mathbb{N}$ is the *initial marking* (token distribution).

A transition $t \in T$ is *enabled* under marking M if every input place p of t holds at least one token: $\forall p \in \bullet t : M(p) \geq 1$. Firing t removes one token from each input place and adds one token to each output place:

$$M'(p) = M(p) - F(p, t) + F(t, p) \quad (2.4)$$

Petri nets provide the foundation for several analysis problems relevant to IoT networks:

Reachability Determine whether a given marking M' is reachable from M_0 .

Boundedness Determine whether the number of tokens in any place is bounded.

Liveness (deadlock-freedom) Determine whether every transition can eventually fire from every reachable marking.

In Aetherium, the Petri net formalism is applied at the *network level*: each automaton’s current state contributes a conceptual token, and the combined network of automata is analyzed for deadlocks, bottlenecks, and resource contention. This is described in detail in Chapter 9.

2.4 Record-and-Replay Debugging

Record-and-replay is a debugging strategy in which program execution is first *recorded* — capturing non-deterministic inputs, scheduling decisions, and event orderings — and then *replayed* deterministically from the recorded log to reproduce the original execution. The technique was formalised by LeBlanc and Mellor-Crummey [11], who applied it to shared-memory parallel programs by logging synchronisation events; their system, *Instant Replay*, demonstrated that deterministic replay is practical even when the original execution involved data races.

Engblom surveys the broader landscape of reverse execution and debugging tools [5], distinguishing three levels of granularity:

Instruction-level Every CPU instruction is recorded; used by full-system simulators. Provides complete fidelity but imposes significant execution overhead.

System-call-level Only OS interactions are recorded; sufficient for most user-space bugs and used by tools such as Mozilla `rr`. Typically 1–2× overhead on x86.

Application-level Only domain-relevant events are recorded (messages, state transitions, variable updates). Lowest overhead; fidelity is limited to the recorded abstraction.

Geels et al. apply application-level replay to distributed systems [6]: messages between components are logged at each replica, and replay re-injects them in the same order, reproducing failures that would otherwise require weeks of manual investigation.

Aetherium adopts an application-level strategy tailored to EFSM networks. Every state transition, variable update, and inter-device message is recorded as a structured `TraceRecord`. The IDE can step backward and forward through this trace. Beyond offline replay, Aetherium implements *live rewind*: the server reconstructs the automaton state at any past timestamp and dispatches a `RestoreState` command to the physical device, resetting it to that historical state without stopping the orchestration layer. This mechanism is described in detail in Section 6.6.

Chapter 3

Analysis of Existing Solutions

Before designing Aetherium Automata, two representative existing platforms were analysed: Node-RED, which represents the flow-based IoT automation category, and Eclipse 4diac, which represents the IEC 61499 function-block category. Together they cover the main points of comparison relevant to the thesis goals: explicit state modelling, embedded portability, testability, and observability.

3.1 Node-RED

3.1.1 Overview

Node-RED [16] is a low-code, flow-based programming tool originally developed in early 2013 by Nick O’Leary and Dave Conway-Jones at IBM’s Emerging Technology Services group. It was open-sourced in September 2013 and later donated to the OpenJS Foundation (through the JS Foundation in 2016). Today it is one of the most widely adopted visual programming environments for IoT prototyping.

3.1.2 Programming Model

Node-RED implements the *flow-based programming* paradigm [15], in which an application is expressed as a directed graph of *nodes* connected by *wires*. Each node encapsulates a single unit of processing and communicates with adjacent nodes by passing JavaScript message objects (`msg`). Flows are the top-level grouping unit and are serialised as JSON. The browser-based editor communicates with a Node.js runtime that executes the flows.

Three kinds of built-in nodes are provided:

Input nodes Inject messages into the flow from external sources — timers, MQTT brokers, HTTP endpoints, or hardware GPIO.

Processing nodes Transform, filter, or route messages — function nodes accept arbitrary JavaScript code, switch nodes branch on message properties.

Output nodes Forward messages to external sinks: MQTT, HTTP, or database.

A community ecosystem of over 5 000 third-party nodes extends the platform with protocol adapters, UI widgets, and cloud service integrations.

3.1.3 State Management

Node-RED has no built-in concept of a state machine. Application-level state must be managed explicitly using *context storage*: node context (local to one node), flow context (shared within a flow tab), or global context (shared across all flows). Accessing and mutating context from within function nodes requires manual bookkeeping and is untyped.

Several community packages attempt to address this gap by adding FSM nodes (e.g. `node-red-contrib-finite-state-machine`), but these are third-party add-ons with no native IDE integration, no visual state graph, and no support for timed or probabilistic transitions. Representing complex stateful logic natively in Node-RED leads to large, hard-to-navigate flow graphs in which control flow and data flow are interleaved.

3.1.4 Embedded Targets and Observability

Node-RED runs on the Node.js runtime, which requires a relatively capable host: minimum a Raspberry Pi class device with a full operating system. Bare-metal microcontrollers such as ESP32 (without an OS) or ARM Cortex-M targets are not directly supported.

Runtime observability is provided through a Debug side-panel that displays message values at selected wire taps. There is no structured execution trace, no replay capability, and no fault injection mechanism. Reproducing a timing-sensitive failure requires either manual test setup or third-party logging infrastructure.

3.1.5 Summary

Node-RED excels at rapid prototyping and data routing in environments where state is simple or absent. It is poorly suited to applications requiring rich stateful behaviour, reproducible testing under adverse conditions, or deployment on constrained embedded hardware.

3.2 Eclipse 4diac

3.2.1 Overview

Eclipse 4diac [4] is an open-source IDE and runtime for IEC 61499-based distributed control. First presented by Strasser et al. [21], it is now an Eclipse Foundation project with a dedicated reference text [25].

3.2.2 IEC 61499 and Function Blocks

IEC 61499 is an international standard for distributed industrial process measurement and control systems. It defines a *Function Block* (FB) as the primary computational unit and extends the earlier IEC 61131-3 PLC programming standard to multi-device, distributed architectures.

Three FB types are defined:

Basic Function Block (BFB) The fundamental executable unit. Its behaviour is defined by an *Execution Control Chart* (ECC) — an event-driven state machine — combined with one or more *algorithms* expressed in Structured Text, Ladder Diagram, or Function Block Diagram (languages drawn from IEC 61131-3). A BFB transitions

between ECC states when input *events* arrive and associated Boolean guard conditions are satisfied. Executing an ECC state fires the associated algorithm, which may produce output events and set output data variables.

Composite Function Block (CFB) A network of interconnected FBs, providing hierarchical composition without a separate ECC. The internal topology is fixed at design time.

Service Interface Function Block (SIFB) Provides a typed interface to external services such as hardware peripherals, network protocols, or operating system calls.

The ECC of a BFB is structurally similar to an EFSM (Section 2.2): states, guarded transitions, and per-state algorithms. However, the trigger mechanism differs fundamentally — transitions fire only in response to named input *events* (discrete signals), not as a result of continuous condition polling or timer expiry. Timed behaviour must be implemented using dedicated timer SIFBs wired into the network, and probabilistic transitions are not part of the standard.

3.2.3 4diac IDE and FORTE Runtime

The **4diac IDE** is based on the Eclipse framework. It provides a graphical editor for composing FB networks, an ECC editor for designing BFB state machines, and deployment tooling for distributing application partitions across multiple devices.

The **4diac FORTE** runtime is a portable C++ implementation of the IEC 61499 execution model targeting 16-bit and 32-bit embedded devices. It supports real-time execution and online reconfiguration of running applications. Supported platforms include Linux, Windows, VxWorks, and several bare-metal embedded targets.

3.2.4 Limitations Relevant to this Thesis

Despite its rigorous foundations, 4diac exhibits several limitations from the perspective of the Aetherium design goals:

- **Event-only trigger model.** The ECC transitions fire exclusively on named input events. There is no native support for time-based transitions (without auxiliary timer FBs), no probabilistic selection, and no direct reactivity to data threshold crossings without additional logic.
- **Scripting.** Algorithms are written in IEC 61131-3 languages (primarily Structured Text). There is no embedded general-purpose scripting language comparable to Lua. Dynamic behaviour that depends on runtime arithmetic requires either pre-compiled C++ SIFBs or complex FB networks.
- **IDE weight.** The Eclipse-based IDE carries a significant installation footprint and a steep learning curve, in contrast to a purpose-built Electron application.
- **No fault injection or replay.** 4diac provides no mechanism to inject network delays, packet loss, or disconnection events into a running deployment, and no execution trace suitable for step-by-step replay.

- **Informal execution semantics.** The original IEC 61499 standard left execution semantics underspecified, leading to interpreter divergence across implementations. While later editions improved this, the ECC execution model remains more complex to reason about than the deterministic tick-based resolution used in Aetherium.

3.3 Comparison

Table 3.1 summarises the key characteristics of both analysed platforms against Aetherium Automata.

Table 3.1: Feature comparison of Node-RED, Eclipse 4diac, and Aetherium Automata.

Feature	Node-RED	4diac	Aetherium
Explicit state model	No	Yes (ECC)	Yes (EFSM)
Transition types	N/A	Event + guard	Classic, Timed, Event, Probabilistic, Immediate
Scripting language	JavaScript	Structured Text	Lua 5.4
Visual editor	Browser (Node.js)	Eclipse IDE	Electron desktop
Embedded runtime	No	Yes (FORTE, C++)	Yes (C++17)
Embedded targets	Raspberry Pi+	16/32-bit MCUs	ESP32, Pico, MCXN947
Program format	JSON	XML (FBT)	YAML
Typed variables	No (JavaScript)	Yes (IEC 61131-3)	Yes (7 types)
Fault injection	No	No	Yes
Time-travel debugging	No	No	Yes
Structural analysis	No	No	Petri net
Target domain	General IoT	Industrial	IoT + embedded

The analysis reveals that neither platform simultaneously satisfies all of the requirements identified in Section 3.4. Node-RED lacks explicit state modelling entirely and cannot run on bare-metal embedded targets. 4diac provides formal state machines through the ECC but restricts transitions to an event-driven model and provides no fault injection or replay capability. Aetherium Automata addresses these gaps by combining a five-type EFSM model, a portable C++ runtime, deterministic fault injection, and execution trace replay within a single unified toolchain.

Programmer’s mental model. Beyond feature checklists, the three platforms differ fundamentally in *what the programmer sees and reasons about*.

In **Node-RED** the primary abstraction is the *data flow*: messages travel between processing nodes along wires. The programmer thinks in terms of message routing — what arrives, what is transformed, where it is forwarded. State is implicit and must be maintained manually in untyped context storage. There is no structural view of the system; the graph *is* the program, and scaling it produces a graph that grows in visual complexity without gaining analytical leverage.

In **Eclipse 4diac** the programmer works at the level of *function block composition*: components are wired together at design time, and the per-component behaviour is written as Structured Text algorithms attached to an ECC. The perspective is closer to traditional software engineering — the ECC is a design aid, but the live system is code executing inside a runtime that is largely opaque to the developer.

In **Aetherium** the programmer operates at three complementary levels simultaneously. At the *behavioural level*, each device runs a named state machine whose current state, variable values, and pending transitions are live-visible in the IDE. At the *network level*, the IDE aggregates all connected devices and their signal exchanges into a single monitoring view. At the *structural level*, the Petri net panel lifts the entire multi-automata system into a higher-dimensional representation where deadlocks, bottlenecks, and resource contention are computed and annotated rather than guessed. This progression — from single-automaton behaviour, through live network state, to structural network analysis — directly realises the Deployment-Aware Development philosophy introduced in Section 1.1.

3.4 Requirements

Based on the analysis above, the thesis goals, and the two design principles introduced in Section 1.1, the following requirements were identified. Requirements R1–R4 form the foundational model and runtime; R5–R7 realise Reality-in-the-Loop; R8–R9 realise Deployment-Aware Development.

- R1 – EFSM expressiveness** The model must support guarded transitions, time-driven transitions, event/signal-driven transitions, probabilistic transitions, and unconditional (epsilon) transitions. Probabilistic transitions are essential for RitL: rare real-world events must be expressible and their frequency tunable without changing the automaton definition.
- R2 – Scripting** Guards and actions must be expressible as scripts evaluated against the current variable store. Lua is chosen as the scripting language.
- R3 – Portability** The runtime engine must compile and execute on desktop Linux/macOS, ESP32 (Arduino/FreeRTOS), Raspberry Pi Pico, and ARM Cortex-M (MCXN947).
- R4 – DSL** Automata definitions must be authored in a human-readable format. YAML is chosen.
- R5 – Real-time monitoring** The IDE must display the current state, variable values, and connection metadata (RTT, battery, connection type) of running automata with low latency. (*DAD*)
- R6 – Fault injection** The system must support configurable fault injection (delay, jitter, drop, disconnect) at the communication boundary to enable controlled RitL testing. (*RitL*)
- R7 – Time-travel debugging** Execution traces must be recorded on the server and replayable step-by-step in the IDE, closing the observe-debug loop for injected faults. (*RitL*)

- R8 – Structural analysis** The system must lift multi-automata networks to Petri nets and detect deadlocks, bottlenecks, and resource contention, giving the developer a network-level analytical view. (*DAD*)
- R9 – Black-box contracts** Automata must be able to declare a formal interface contract (ports, observable states, events) that is visible to monitors and to the fault injector without exposing internal implementation. (*RitL + DAD*)

Chapter 4

System Architecture

The Aetherium Automata platform is composed of four major components that form a layered architecture:

1. **Engine** — a portable C++17 library that parses, loads, and executes EFSM definitions.
2. **Server** — an Elixir/OTP application that manages connected devices, routes messages, and aggregates state.
3. **Gateway** — an Elixir/Phoenix application that exposes a WebSocket channel interface to the IDE.
4. **IDE** — an Electron desktop application (TypeScript/React) for visual design, deployment, and monitoring.

Figure 4.1 shows the high-level component topology.

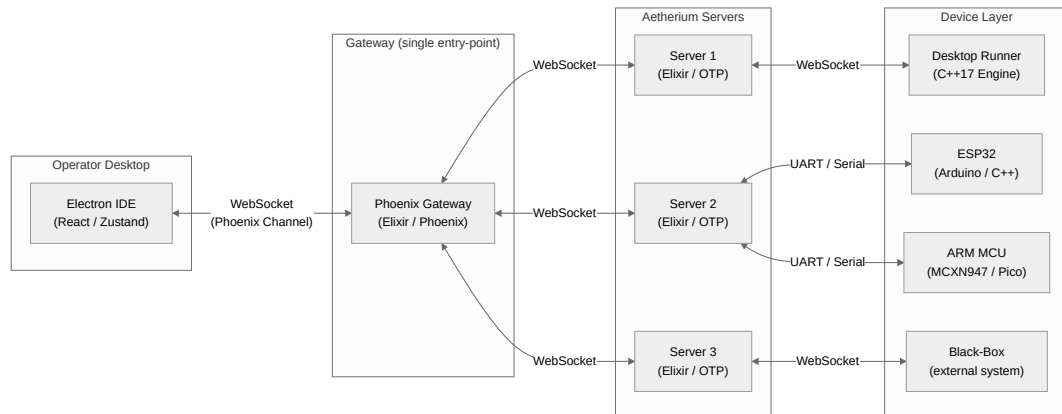


Figure 4.1: High-level component architecture of Aetherium Automata.

4.1 Technology Selection Rationale

Every major technology choice in Aetherium is a direct consequence of the two design principles introduced in Section 1.1. This section explains why each component uses the language and runtime that it does.

4.1.1 Elixir and the BEAM for the Server and Gateway

The server and gateway are written in Elixir, which compiles to bytecode for the BEAM virtual machine — the same runtime that underpins Erlang [1]. The choice is deliberate and has concrete implications for a fleet-management backend.

Process-per-device isolation. BEAM processes are not OS threads; they are extremely lightweight green threads managed by the VM. A freshly spawned process consumes approximately 2–4 KB of heap. In practice this means that each connected device can be represented by its own process (a GenServer holding that device’s state), and the fleet can scale to thousands of devices on a single server node without the shared-memory overhead or lock contention that would accompany the same design in a thread-per-device model.

OTP supervision trees and let-it-crash. The Open Telecom Platform (OTP) provides a set of behavioral abstractions — GenServer, Supervisor, Registry — that impose a structured message-passing discipline on concurrent code. Every device process is supervised: if a device handler encounters an unexpected message or a protocol violation, it crashes and is restarted in isolation by its supervisor. The remaining devices are unaffected. This *let-it-crash* philosophy replaces defensive `try/catch` blocks with a structurally enforced recovery path, making the server resilient to individual device misbehavior by construction [1].

Preemptive scheduling and throughput. BEAM uses a preemptive reduction-counting scheduler: each process is allotted a fixed number of reductions (roughly equivalent to function calls) before it is descheduled. A single misbehaving handler cannot starve other processes, which is essential when one slow device must not delay telemetry from fifty healthy ones. The scheduler’s cooperative multitasking analogue in most IoT backends is the event loop, where a blocking call freezes the entire server. BEAM makes blocking impossible by design.

Hot code reloading. BEAM supports atomic module replacement at runtime: a new version of the gateway code can be loaded while WebSocket connections are live, and all new message arrivals are dispatched to the new code without dropping any in-flight state. For a production fleet-management system this is critical; for a development-phase system it is convenient. Either way it removes the need for a restart-based deploy cycle.

4.1.2 C++ for the Engine

The engine is the component that runs on devices, which span an extreme range: from a quad-core Linux desktop down to a dual-core Cortex-M0+ microcontroller (Raspberry Pi Pico) or a dual-core Xtensa LX6 (ESP32). This range rules out languages with managed runtimes or non-trivial standard library footprints. C++17 with disciplined allocator usage is the only mainstream language that can target all of these platforms with a single source tree.

Engine-as-library design. The engine is packaged as a static library rather than an application. The consuming project provides `main()`, hardware initialization, and the platform-specific implementations of three abstract interfaces. The engine itself contains

no `main()`, no global state beyond what the application registers, and no compile-time assumption about the OS or threading model. This means the same engine code can be linked into:

- a desktop process that opens a WebSocket connection and runs the tick loop in a thread,
- an ESP32 Arduino sketch that calls `engine.tick()` from its FreeRTOS task,
- a Raspberry Pi Pico application that drives the tick loop from a hardware timer interrupt.

Abstract platform interfaces. The three interfaces that isolate platform-specific behavior are:

IClock Monotonic time source. The desktop implementation delegates to `std::chrono::steady_clock`. Embedded implementations read a hardware timer peripheral. The interface returns a 64-bit microsecond count; the engine uses this for timed-transition deadlines and fault injection timing.

IRandomSource Pseudorandom number generator. The desktop implementation uses `std::mt19937_64` seeded from `/dev/urandom`. Embedded implementations use the hardware RNG peripheral when available, or a software LFSR. Probabilistic transitions draw from this source on every evaluation.

IScriptEngine Script evaluation. The desktop implementation uses Lua 5.4 via Sol2 [22], providing full scripting capability. Embedded implementations can substitute a lightweight expression evaluator or a no-op stub. When the no-op is used, guards reduce to the static values defined in the YAML, and the resulting binary is approximately 64 KB.

CMake build target selection. The top-level `CMakeLists.txt` defines two interface targets: `aetherium_engine_desktop` and `aetherium_engine_embedded`. Linking against the former pulls in the Lua, IXWebSocket [12], and RapidYAML [3] dependencies. Linking against the latter pulls in only the embedded stubs. Platform-selection variables (`AETHERIUM_TARGET=esp32`, `AETHERIUM_TARGET=pico`, `AETHERIUM_TARGET=desktop`) configure the right target automatically.

4.2 Component Responsibilities

4.2.1 Engine

The engine is the core library. It is written in C++17 with no dynamic dependencies beyond the platform-specific backends (see Section 4.1). It is responsible for:

- parsing automata definitions from YAML [24] (via RapidYAML [3]),
- maintaining the variable store and evaluating Lua guards and actions (via Sol2 [22]),
- resolving and firing transitions on each execution tick,
- recording the execution trace for time-travel debugging,
- transmitting telemetry and receiving commands over the pluggable transport interface.

4.2.2 Server

The server is an Elixir/OTP application structured around a supervised process tree. It maintains the device registry as an OTP Registry, where each entry maps a device identifier to the PID of its dedicated GenServer process. The device GenServer holds the last-known state snapshot, a buffer of unacknowledged outbound messages, and the device’s connection metadata (transport type, RTT, battery level). When a device disconnects, its GenServer is not immediately terminated; it enters a reconnection-wait state so that deployment state survives transient network interruptions.

The server exposes an internal API consumed by the gateway. Incoming IDE commands (deploy, start, stop, set input, inject fault) are translated into GenServer calls. Outgoing telemetry events are forwarded to a Phoenix PubSub topic to which the gateway subscribes.

4.2.3 Gateway

The gateway is an Elixir/Phoenix application that exposes a single *Phoenix Channel* endpoint at `/socket/websocket`. The IDE connects to this endpoint over a persistent WebSocket. The gateway acts as a protocol translator: it maps the IDE’s JSON-over-WebSocket commands to internal server calls, and it relays the PubSub telemetry stream back to the IDE as structured channel events. The separation between server (domain logic) and gateway (transport/protocol) means that a REST API or CLI tool could consume the same server without touching the gateway.

4.2.4 IDE

The IDE is an Electron desktop application. The main process handles native file system access and exposes file I/O via IPC channels. The renderer process is a React single-page application with Zustand-based state management. Communication with the gateway occurs exclusively through a `PhoenixGatewayService` singleton that manages connection lifecycle, channel joins, and automatic reconnection.

4.3 Component Interaction and Data Flow

Figure 4.1 shows the static topology; the dynamic interaction sequence is as follows.

1. The IDE loads a project file and the developer clicks *Deploy*. The IDE serializes the YAML automaton definitions and sends them as a `deploy` channel event to the gateway.
2. The gateway forwards the payload to the server, which stores the definitions in the target device’s GenServer and pushes a binary `DEPLOY` frame to the device over its WebSocket connection.
3. The engine on the device receives the frame, parses the definitions, and enters the *running* state. On every tick it evaluates transitions and emits a `TELEMETRY` frame containing the current state, variable values, and timing metadata.
4. The server receives the telemetry frame, updates the device GenServer, and publishes the event to the PubSub topic.

5. The gateway relays the event to all subscribed IDE channel sessions. The IDE updates the relevant Zustand store slice; connected React panels re-render automatically.

This pipeline is entirely event-driven: no polling occurs anywhere in the stack. The BEAM scheduler ensures that a burst of telemetry from one device does not delay messages for others.

4.4 Deployment Model

Devices are deployed using Docker Compose for local multi-service orchestration. A typical local stack consists of the gateway service, the server service, one or more device emulator services (each running the engine as a desktop binary), and optionally a black-box probe container for sandboxed external systems. Real embedded targets (ESP32, Pico) connect to the same gateway over the network; from the server’s perspective a real device and an emulated one are indistinguishable—both speak the same binary wire protocol over WebSocket.

Chapter 5

Automata Model and DSL

5.1 Model Overview

An *automaton* in Aetherium is a named, versioned EFSM definition. At the top level it contains:

- **config** — name, version, layout type (**inline** or **folder**), description, author, tags,
- **variables** — typed I/O and internal variables,
- **automata** — the EFSM body: initial state, states, and transitions.

5.2 States

Each state $q \in Q$ is identified by a string **StateId**: a letter or underscore followed by zero or more alphanumeric characters or underscores. States carry three optional Lua code hooks:

on_enter Executed once when the machine transitions into this state.

body Executed on every execution tick while the machine remains in this state.

on_exit Executed once immediately before transitioning away.

One state per automaton is designated the **initial_state**; the engine enters it when execution begins.

5.3 Variables

Variables form the data context V of the EFSM. Each variable is defined by a *VariableSpec* that carries:

- **name** — unique string identifier,
- **type** — one of **bool**, **int32**, **int64**, **float32**, **float64**, **string**, **binary**, **table**,
- **direction** — **input** (read-only, written by the external environment), **output** (write-only for the environment, written by the automaton), or **internal** (read-write by the automaton),

- **default** — optional initial value.

At runtime each variable is a *Variable* instance wrapping a type-safe **Value** variant and a boolean **changed()** flag. The flag is set on every write and cleared explicitly or on state transition. It is used by event-type transitions for $O(1)$ reactive detection without polling.

5.4 Transition Types

Aetherium defines five transition kinds, each with distinct enabling semantics:

5.4.1 Classic

A classic transition is enabled when a Lua guard expression evaluates to **true**:

$$\text{enabled}(t) \iff g_t(V) = \text{true} \quad (5.1)$$

Guards are arbitrary Lua expressions operating on the variable store through the engine API.

5.4.2 Timed

A timed transition fires based on time domain conditions. Five modes are supported:

after Fire after a fixed delay Δt (in milliseconds) from the last state entry.

every Fire repeatedly with period τ .

timeout Fire if no other transition from the same state has fired within Δt .

at Fire at an absolute timestamp.

window Fire only if the current time lies within $[t_{start}, t_{end}]$.

An optional jitter parameter $\varepsilon \geq 0$ adds a uniform random offset to the scheduled fire time:

$$t_{fire} = t_{target} + \mathcal{U}(0, \varepsilon) \quad (5.2)$$

This prevents synchronized firing across multiple automata instances and models real-world timer imprecision.

5.4.3 Event

An event transition is enabled when a named variable satisfies a specified signal condition. Supported trigger types are:

on_change Any write to the variable since the last evaluation.

on_rise The variable transitions from **false** to **true**.

on_fall The variable transitions from **true** to **false**.

on_threshold The variable crosses a comparator threshold: $v \bowtie k$ where $\bowtie \in \{>, <, \geq, \leq, =, \neq\}$.

An optional debounce delay δ_d filters spurious signals shorter than the threshold. The **one_shot** flag causes a threshold trigger to fire at most once per crossing. A transition may specify multiple triggers, combined with either AND or OR logic (**require_all**).

5.4.4 Probabilistic

A probabilistic transition is always structurally enabled. Its *weight* $w_i \in [0, 10000]$ (representing 0.00%–100.00% in fixed-point) participates in a weighted roulette selection among all enabled transitions in the same priority group. The selection probability is:

$$P(T_i) = \frac{w_i}{\sum_{j \in G} w_j} \quad (5.3)$$

where G is the set of enabled transitions in the highest-priority group. Weights may be Lua expressions re-evaluated on each tick, enabling dynamic probability distributions.

5.4.5 Immediate (Epsilon)

An immediate transition is always enabled and is resolved before transitions of any other type. It models the ε -transitions of non-deterministic finite automata: unconditional, instantaneous state changes used to chain states without observable intermediate steps.

5.5 Transition Resolution Algorithm

On each execution tick the engine resolves which transition, if any, fires from the current state. Algorithm 1 gives the full procedure.

Algorithm 1 Transition resolution on a single execution tick.

```

1: procedure TICK( $q, vars$ )
2:    $T_{out} \leftarrow$  all outgoing transitions from state  $q$ 
3:    $E \leftarrow \{t \in T_{out} \mid \text{ENABLED}(t, vars)\}$  ▷ evaluate guards, timers, events
4:   if  $E = \emptyset$  then
5:     execute body hook of  $q$ 
6:     return
7:   end if
8:    $p^* \leftarrow \min_{t \in E} t.priority$ 
9:    $G \leftarrow \{t \in E \mid t.priority = p^*\}$  ▷ top-priority group
10:  if  $|G| = 1$  then
11:     $t^* \leftarrow$  the sole element of  $G$ 
12:  else if  $\forall t \in G : t.weight > 0$  then
13:     $t^* \leftarrow \text{WEIGHTEDSAMPLE}(G)$  ▷ Eq. (5.3)
14:  else
15:     $t^* \leftarrow$  first element of  $G$  in declaration order
16:  end if
17:  execute on_exit hook of  $q$ 
18:  execute body hook of  $t^*$ 
19:   $q' \leftarrow \delta_E(q, t^*)$ 
20:  execute on_enter hook of  $q'$ 
21:  return  $q'$ 
22: end procedure

```

Priority establishes a total deterministic order; probabilistic selection applies only within a single priority group. This cleanly separates intentional non-determinism from deterministic control flow.

5.6 Black-Box Contract

An automaton may optionally declare a *black-box contract* — a formal interface specification for external systems. The contract defines:

- **ports** — named inputs/outputs with direction, type, observability flag (visible to monitors), and fault-injectability flag,
- **emitted events** — names of events the automaton produces,
- **observable states** — the subset of states visible to external monitors,
- **resources** — shared or exclusive resources with capacity and latency sensitivity annotations.

Black-box contracts are used by the structural analysis layer (Chapter 9) and by the fault injection subsystem.

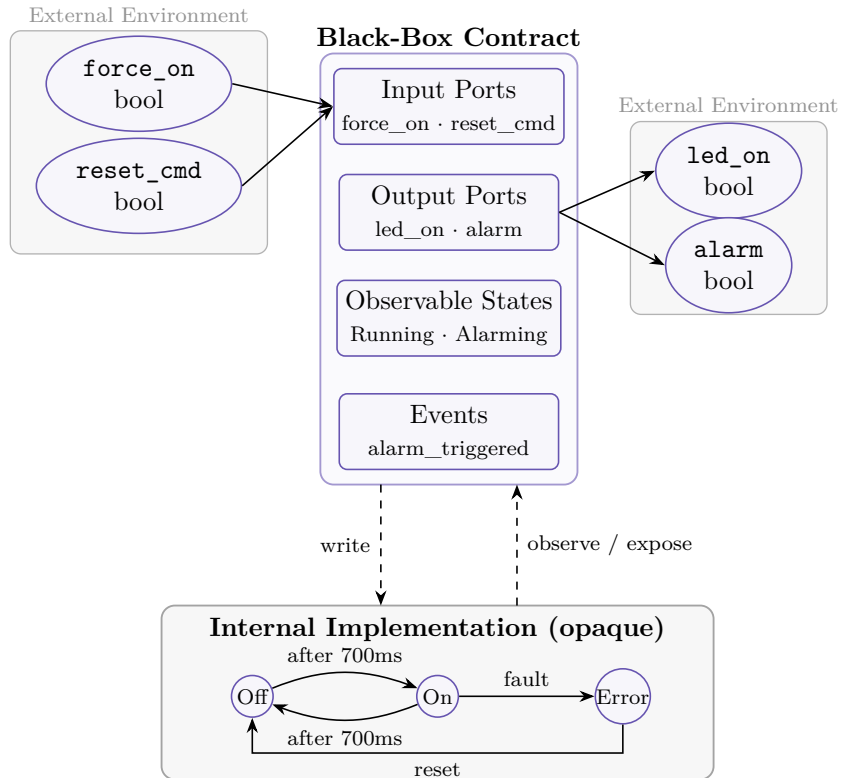


Figure 5.1: Black-box contract boundary. The visible contract layer exposes named input/output ports, observable states, and emitted events to the external environment. The internal implementation — states, variables, and transitions — is opaque.

5.7 YAML Domain-Specific Language

Automata are authored in YAML [24]. The top-level schema is:

```
1 version: 0.0.1
2 config:
3   name: <string>
4   type: inline | folder
5   location: <path> # required if type: folder
6   language: lua # optional, default lua
7   description: <string> # optional
8   author: <string> # optional
9   tags: [<string>, ...] # optional
10
11 variables:
12   - name: <VarName>
13     type: bool | int32 | int64 | float32 | float64 | string | binary
14     direction: input | output | internal
15     default: <value> # optional
16
17 automata:
18   initial_state: <StateId>
19   states:
20     <StateId>:
21       on_enter: |
22         <lua code>
23       body: |
24         <lua code>
25       on_exit: |
26         <lua code>
27   transitions:
28     <TransitionId>:
29       from: <StateId>
30       to: <StateId>
31       type: classic | timed | event | probabilistic | immediate
32       priority: <int> # lower value = higher priority
33       weight: <float> # for probabilistic selection
34       condition: | # for type: classic
35         <lua expression>
36       timed: # for type: timed
37         mode: after | at | every | timeout | window
38         after: <ms>
39         jitter: <ms> # optional
40       event: # for type: event
41         triggers:
42           - signal: <VarName>
43             trigger: on_change | on_rise | on_fall | on_threshold
44             threshold:
45               operator: ">" | "<" | ">=" | "<=" | "==" | "!="
46               value: <number>
47             one_shot: <bool>
48             require_all: <bool>
```

Listing 5.1: YAML DSL top-level structure.

5.7.1 Layout Types

The `inline` layout places all state and transition Lua code directly inside the YAML file. The `folder` layout externalizes code into separate files referenced by path, enabling better IDE tooling, version-control diffs, and code reuse.

5.7.2 Validation Rules

The parser enforces the following constraints:

- All state and transition identifiers must follow the identifier naming convention (letter-or-underscore start, alphanumeric continuation).
- Every `from/to` endpoint must name a state defined in `automata.states`.
- Variable names must be unique within an automaton.
- Exactly one state must be designated `initial_state`.
- The `timed.mode` field determines which time parameters are required.

Chapter 6

Runtime Engine

6.1 Execution Tick Cycle

The engine runs a continuous tick loop. On each tick it advances timers, evaluates transition guards, selects and fires at most one transition per automaton, and emits telemetry. Figure 6.1 shows the full cycle.

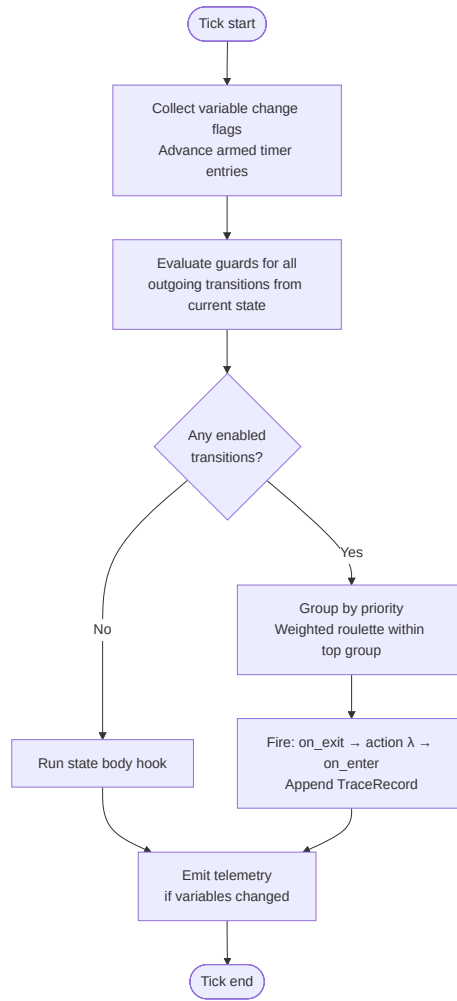


Figure 6.1: Engine execution tick cycle. A transition fires at most once per tick per automaton.

6.2 Platform Abstraction

All platform-specific behavior is isolated behind three abstract interfaces, allowing the same runtime core to compile on desktop Linux and bare-metal microcontrollers without `#ifdef` sprawl. Figure 6.2 shows the two-layer structure.

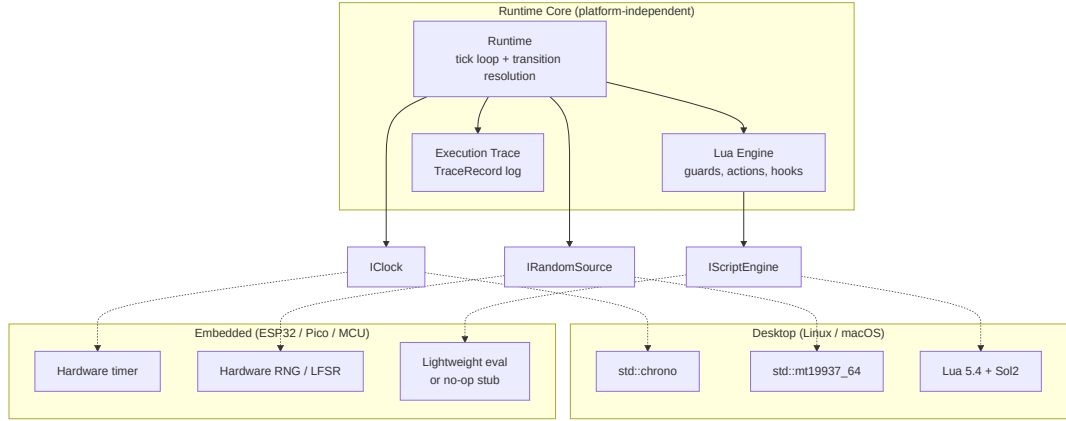


Figure 6.2: Platform abstraction layer. The runtime core depends only on the three interfaces; concrete implementations are selected at link time.

The interfaces are: **IClock** (monotonic time source), **IRandomSource** (seeded PRNG for reproducible fault injection and probabilistic transitions), and **IScriptEngine** (Lua 5.4 [9] via Sol2 [22] on desktop; a lightweight evaluator or no-op stub on constrained targets). CMake build targets select the concrete implementations at compile time via RapidYAML [3] and IXWebSocket [12] as the only external dependencies on desktop.

6.3 Lua Scripting

Guards and state/transition code are authored in Lua 5.4 [10, 9]. The engine exposes the following global API functions to scripts:

value(name) Read the current value of a variable by name.

setVal(name, v) Write a value to a variable by name.

check(name) Return **true** if the variable’s **changed()** flag is set.

emit(name) Signal a named event.

log(msg) Write a message to the telemetry log.

rand() Return a uniform random float in $[0, 1)$ using the platform **IRandomSource**.

now() Return the current monotonic timestamp in milliseconds from **IClock**.

clamp(v, lo, hi) Clamp v to $[lo, hi]$.

On platforms where full Lua is not available (highly constrained microcontrollers), a **SimpleScriptEngine** or no-op stub can be substituted via the **IScriptEngine** interface without modifying any other engine module.

6.4 Timer Management

The runtime maintains a table of timer entries, one per timed transition. Each entry stores:

- `startTime` — the monotonic time at which the timer was armed,
- `targetTime` — the computed fire time (including jitter),
- `repeatCount` — remaining repetitions for `every` mode (0 = infinite).

On each tick the runtime queries `IClock::now()` and compares it against `targetTime` for each armed timer. Expired timers with `mode = every` are immediately re-armed with a new `targetTime = now + period + jitter`.

6.5 Fault Injection

Fault injection is a proven technique for testing the resilience of distributed systems under realistic adverse conditions [2]. A `FaultProfile` can be attached to a deployment descriptor per device. The profile specifies:

Ingress faults Applied to messages arriving at the device:

- fixed delay and jitter (ms),
- drop probability $p_d \in [0, 1]$,
- duplicate probability $p_{dup} \in [0, 1]$.

Egress faults The same parameters applied to outgoing messages.

Disconnect simulation Periodic forced disconnection windows of configurable duration.

Battery drain Per-tick and per-message energy consumption, combined with a battery profile that models charge state and cutoff voltage.

All random decisions within the fault injector use the platform `IRandomSource` seeded from a field in the `FaultProfile`, ensuring deterministic and reproducible fault scenarios across runs.

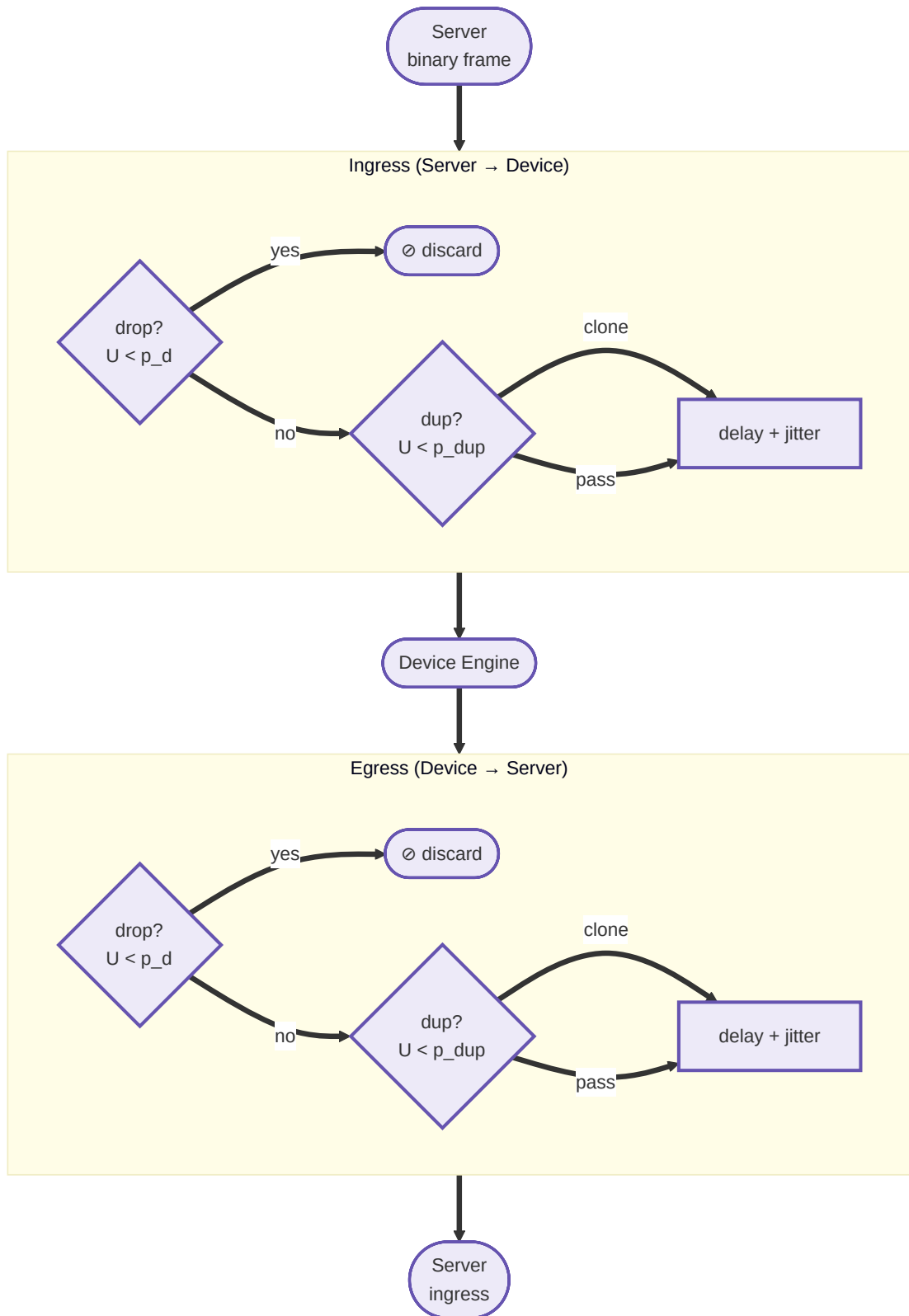


Figure 6.3: FaultProfile message pipeline. Each frame passes through drop, duplicate, and delay decisions independently for ingress (server → device) and egress (device → server). All probabilistic decisions draw from a seeded `IRandomSource`, making fault scenarios fully reproducible.

6.6 Execution Trace and Time-Travel Debugging

Record-and-replay is a well-established strategy for debugging distributed systems: events are recorded during execution and can be replayed deterministically to reproduce failures [6]. Aetherium applies this principle at the automata level. Every engine event is recorded in an `ExecutionTrace` as a sequence of `TraceRecord` entries. Each record carries:

- sequence number,
- wall-clock and monotonic timestamps,
- event type (state transition, variable change, message send/receive),
- old and new values,
- latency measurements,
- the fault actions applied to the associated message (if any).

The trace is exported to newline-delimited JSON (JSON Lines) for post-mortem tooling. The IDE’s runtime view panel can load a trace and step forward and backward through it, reconstructing the exact variable and state values at each recorded instant. This enables deterministic replay of timing-sensitive bugs in distributed systems without requiring hardware reproduction of the fault.

Server-Side State Reconstruction and `RestoreState`

Stepping back in time in the IDE triggers a **full rewind** of the physical device, not merely a visual replay. The implementation spans three layers:

1. **Server-side time series.** An observability module records every `state_changed` and `variable_updated` event forwarded from the device into a per-deployment time series, together with periodic full-state snapshots. The `replay_state_at` function reconstructs the complete automaton state at any past timestamp by finding the nearest earlier snapshot and replaying the events that follow it.
2. **`RestoreState` protocol message (0x48).** A new message type was added to the Automata Plane of the binary wire protocol (range 0x40–0x7F). The message carries the target state name and a snapshot of all variable values as a compact TLV-encoded list. On the Elixir side, `EngineProtocol.encode(:restore_state, %{...})` encodes the frame; `DeviceTransport.send_message/3` delivers it over the existing WebSocket transport.
3. **`Runtime::restoreState` in the C++ engine.** Upon receiving a `RestoreState` frame the engine handler pauses the tick loop, directly sets `ctx_.currentState` and applies the variable snapshot via `ctx_.variables.setValue`, cancels all pending timers, and leaves the engine in `Paused` state awaiting a `Resume` command. No `onEnter` hooks are fired — the state is restored atomically without side-effects.

The gateway channel `rewind_deployment` event orchestrates the flow: the server reconstructs the state at the requested timestamp, broadcasts a `deployment_status` update to connected IDE clients, then dispatches `RestoreState` to the physical device. End-to-end

testing confirms that after a `rewind_deployment` call the device restarts execution from the correct historical state, as evidenced by the first transition fired being the one consistent with the reconstructed variable values.

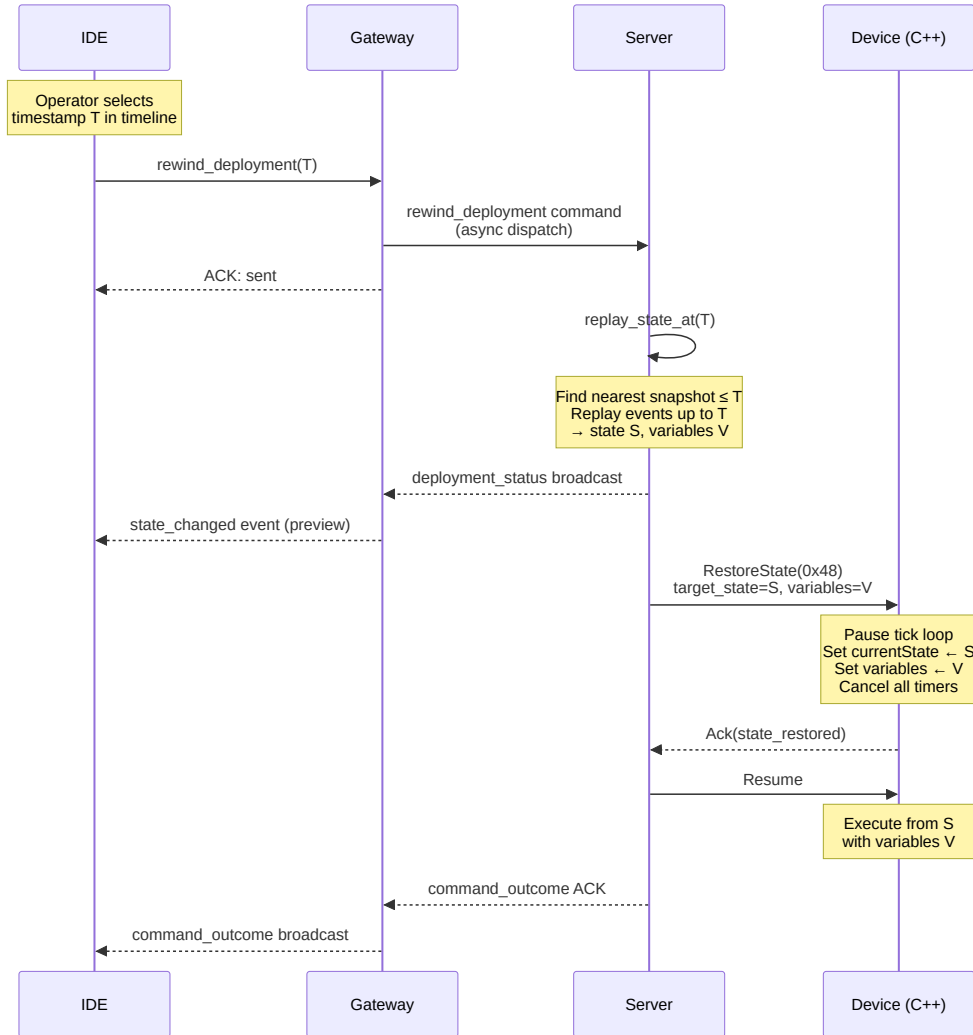


Figure 6.4: Time-travel rewind sequence. The IDE dispatches `rewind_deployment` to the gateway; the server reconstructs the automaton state at timestamp T via `replay_state_at` and delivers a `RestoreState(0x48)` binary command to the device, which pauses, applies the state snapshot atomically, and resumes.

Chapter 7

Gateway and Communication Protocol

7.1 Binary Wire Protocol

All messages between the engine and the server use a compact binary envelope shown in Figure 7.1.

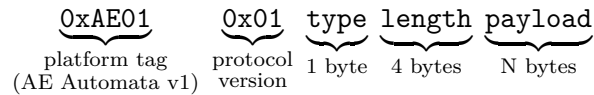


Figure 7.1: Binary message envelope. The 2-byte platform tag 0xAE01 identifies the Aethereum Automata protocol family (v1); the following protocol version byte allows the same platform to evolve its wire format independently.

The protocol is inspired by MQTT’s lightweight, topic-structured publish/subscribe model, but is implemented as a custom binary format rather than using the MQTT standard directly. The custom wire format was necessary to provide structured support for time-travel debugging: the server tags each message with a monotonic sequence number and stores a full execution trace, enabling deterministic replay and **RestoreState** rewinding — capabilities that a generic MQTT broker cannot provide without external instrumentation.

Messages are organized into three categories:

Control plane Hello, Ping, Pong, Status — connection management.

Automata plane LoadAutomata, Start, Stop, Restart — lifecycle commands.

Data plane Input, Output, StateChange, Telemetry — runtime data exchange.

Large automata definitions that exceed the message MTU are fragmented using a chunked-transfer sub-protocol before being reassembled on the receiver side.

7.2 Phoenix Gateway

The gateway is implemented as an Elixir [23]/Phoenix [19] application using Phoenix Channels. Phoenix Channels provide persistent, topic-routed bidirectional WebSocket connections with presence tracking. Each IDE client session maps to one channel subscription.

The gateway exposes the following operations to the IDE:

- `ping()` — test channel connectivity,
- `listDevices()` — retrieve the current device registry,
- `deployAutomata(yaml, deviceId)` — transmit an automaton definition to a device,
- `startAutomata(deviceId, runId)` — command a device to begin execution,
- `stopAutomata(deviceId, runId)` — command a device to halt,
- `setInput(deviceId, varName, value)` — write an input variable,
- `getSnapshot()` — retrieve the current full device/variable state.

Snapshot caching reduces reconnection latency: when the IDE reconnects, it receives the last cached snapshot rather than waiting for a full device re-advertisement cycle.

7.3 End-to-End Message Flow

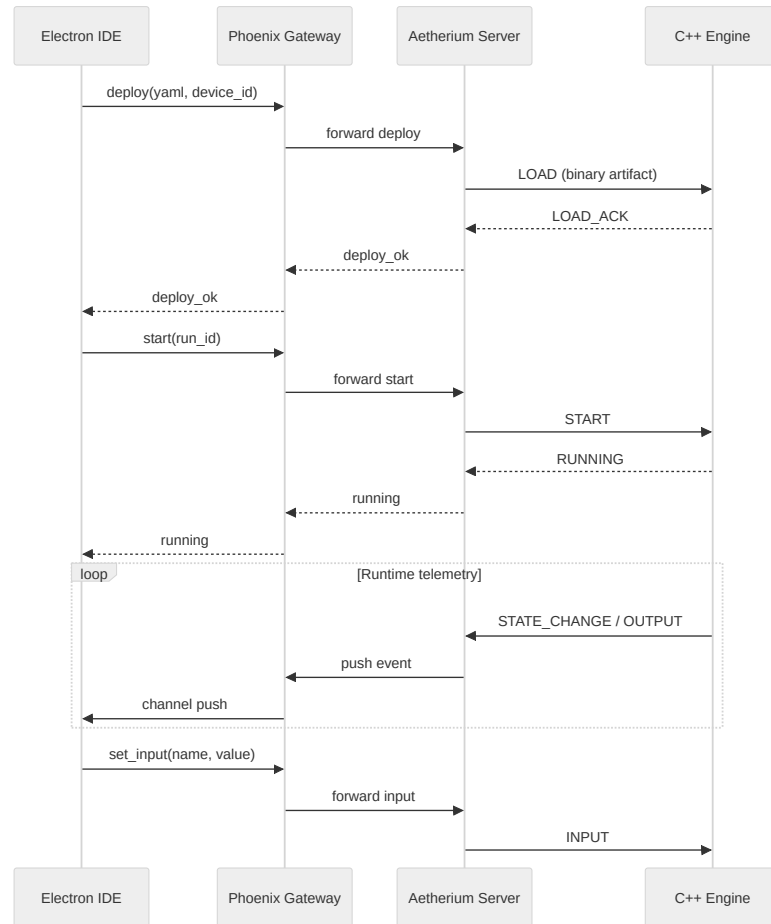


Figure 7.2: End-to-end message sequence for deploy, start, and runtime I/O exchange.

Chapter 8

Integrated Development Environment

8.1 Architecture

The IDE is built on Electron [17], which embeds a Node.js main process and a Chromium-based renderer process. The main process exposes native file system access and IPC bridges. The renderer process is a React [13] single-page application.

State management uses Zustand [20], a lightweight library that exposes individual atomic stores with direct mutation. Each store covers a narrow concern:

Table 8.1: Zustand stores in the IDE renderer.

Store	Responsibility
<code>automataStore</code>	Current automaton definition being edited
<code>projectStore</code>	Multi-automaton project and file management
<code>runtimeViewStore</code>	Live visualization state (current state, variable values, zoom)
<code>gatewayStore</code>	Phoenix channel connection, device list, pending commands
<code>analyzerStore</code>	Petri net analysis results (deadlocks, bottlenecks)

8.2 Phoenix Gateway Service

`PhoenixGatewayService` is the IDE-side client for the Phoenix Channel. It manages the WebSocket lifecycle (connect, reconnect, presence), serializes commands into channel push events, and dispatches received events to the appropriate Zustand store actions. Command outcome tracking allows the UI to display pending/success/error states per operation.

8.3 Panels

The IDE UI is organized into panels:

Canvas / State Machine Editor Visual graph editor. States are rendered as nodes; transitions as directed edges annotated with type, priority, and weight. Users can drag to create states, draw transitions, and edit properties inline.

State Inspector Edits `on_enter`, `body`, and `on_exit` Lua code for the selected state with syntax highlighting.

Transition Inspector Edits condition, type, priority, weight, and type-specific parameters (timed mode, event triggers) for the selected transition.

Variables Panel Unified view of all input, output, and internal variables with current values and types.

Gateway Panel Connection settings, device list, and manual command testing.

Runtime Monitor Panel Live display of the current state (highlighted in the canvas) and variable values during execution.

Log Panel Streaming telemetry event log with filtering by level and source.

Network Panel Topology view of the multi-device network and inter-automaton wiring.

Analyzer Panel Petri net analysis output: bottleneck warnings, deadlock candidates, contention reports.

Petri Net Panel Visual representation of the Petri net derived from the automata network.

Explorer Panel File-based project navigator for folder-layout automata.

Chapter 9

Petri Net Analysis

Petri nets [14] are a well-studied formalism for modeling and analyzing concurrent systems with shared resources. Their graphical nature and formal semantics make them well-suited as a structural analysis layer above a network of communicating automata. This chapter describes how Aetherium lifts an automata network to a Place/Transition (P/T) net, how individual nets are composed into a single model of the full system, what analysis operations the IDE exposes, and how two showcase scenarios exercise these capabilities.

9.1 Formal Definitions

Automaton. An Aetherium automaton is a tuple $\mathcal{A}_i = (S_i, s_i^0, T_i, V_i, G_i, Act_i, \Sigma_i^{\text{out}}, \Sigma_i^{\text{in}})$ where:

- S_i is a finite set of states, $s_i^0 \in S_i$ is the initial state,
- $T_i \subseteq S_i \times S_i$ is the set of transitions,
- V_i is the variable store (typed name–value pairs),
- $G_i : T_i \rightarrow \text{Bool}$ is the guard function evaluated by the script engine,
- $Act_i : T_i \rightarrow \text{Void}$ is the action function executed on firing,
- Σ_i^{out} is the set of named output signal channels this automaton writes to,
- Σ_i^{in} is the set of named input signal channels this automaton reads from.

P/T net. A Place/Transition net is a tuple $\mathcal{N} = (P, T, F, W, M^0)$ where P and T are disjoint finite sets of *places* and *transitions*, $F \subseteq (P \times T) \cup (T \times P)$ is the flow relation, $W : F \rightarrow \mathbb{N}_{>0}$ assigns arc weights, and $M^0 : P \rightarrow \mathbb{N}_{\geq 0}$ is the initial marking.

A transition $t \in T$ is *enabled* under marking M iff for every pre-place p with $(p, t) \in F$: $M(p) \geq W(p, t)$. Firing t produces marking M' where $M'(p) = M(p) - W(p, t) + W(t, p)$, with W extended to return 0 for non-existent arcs.

9.2 Lifting a Single Automaton

The *lifting* of automaton $\mathcal{A}_i$ to a subnet $\mathcal{N}_i = (P_i, T'_i, F_i, W_i, M_i^0)$ is defined as follows.

Places. Each state $s \in S_i$ becomes a place: $P_i = S_i$. Additionally, each output channel $\sigma \in \Sigma_i^{\text{out}}$ yields a dedicated *channel place* p_σ . The channel places are shared with the subnets of automata that list σ in their Σ^{in} .

Transitions and arcs. Each automaton transition $(s_a, s_b) \in T_i$ yields a Petri net transition t_{ab} with:

- a pre-arc (p_{s_a}, t_{ab}) consuming the token from the source state place,
- a post-arc (t_{ab}, p_{s_b}) depositing a token in the target state place.

If the automaton transition also writes to a signal channel $\sigma \in \Sigma_i^{\text{out}}$, an additional post-arc (t_{ab}, p_σ) is added. If the transition guard requires a token on an input channel $\sigma \in \Sigma_i^{\text{in}}$, an additional pre-arc (p_σ, t_{ab}) is added and the token is consumed when the transition fires.

All arc weights are 1 in this basic lifting. Bounded-capacity resources (declared in black-box contracts) are represented by mutual-exclusion places with initial marking equal to the resource capacity.

Initial marking. $M_i^0(s_i^0) = 1$ for the initial state; $M_i^0(s) = 0$ for all other state places; $M_i^0(p_\sigma) = 0$ for all channel places (channels start empty).

Figure 9.1 illustrates the construction for two automata communicating through a single named channel.

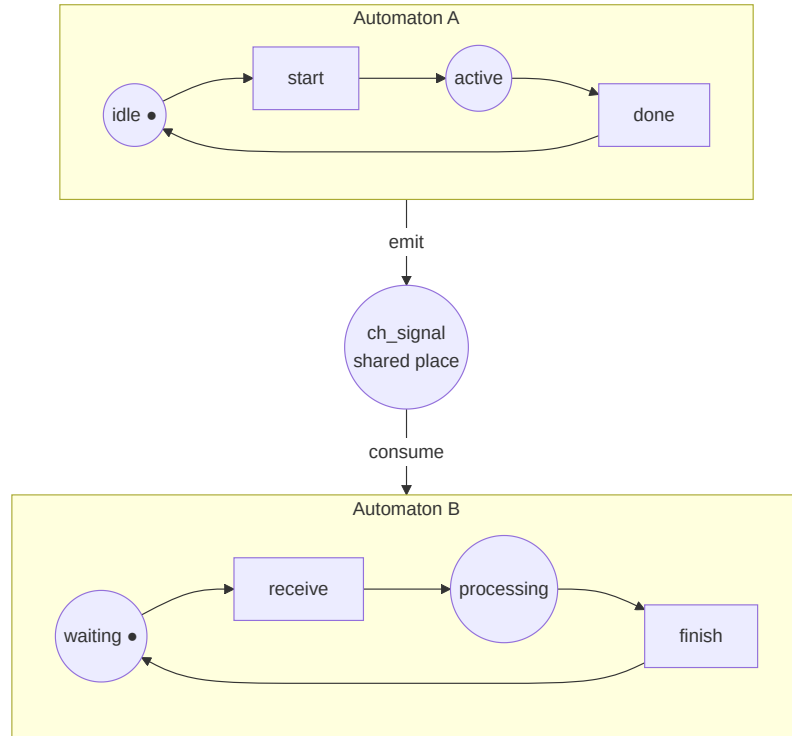


Figure 9.1: Two automata (A and B) lifted to Petri net subnets. The shared channel place `ch_signal` connects the output transition of automaton A to an input transition of automaton B.

9.3 Network Composition

Given a project containing automata $\mathcal{A}_1, \dots, \mathcal{A}_n$, the composed network net $\mathcal{N}$ is constructed by taking the disjoint union of all subnets and identifying shared channel places:

$$\mathcal{N} = \left(\bigcup_i P_i \cup C, \bigcup_i T'_i, \bigcup_i F_i, W, M^0 \right)$$

where $C = \bigcup_i \Sigma_i^{\text{out}}$ is the set of channel places (each appearing once regardless of how many automata reference it), and M^0 is the union of the individual initial markings.

The composition is purely *structural*: it requires no execution trace, only the static YAML definitions. This means the analyzer can report structural hazards before any device is connected.

9.4 Analysis Operations

The analyzer implements four structural checks on the composed net $\mathcal{N}$.

9.4.1 Reachability and Deadlock Detection

The analyzer performs a bounded BFS/DFS over the reachability graph $\mathcal{R}(\mathcal{N}, M^0)$. A marking $M \in \mathcal{R}$ is reported as a *deadlock* if no transition in $\mathcal{N}$ is enabled under M . In an automata network, a deadlock marking corresponds to a tuple of states $(s_1, \dots, s_n)$ from which the system cannot make further progress regardless of input. The analyzer reports both the deadlock marking and the shortest path from M^0 that leads to it.

Because the reachability graph can grow exponentially with the number of automata, the analyzer applies a configurable depth bound. For typical showcase networks (up to 6 automata, up to 8 states each) full enumeration is tractable.

9.4.2 Boundedness

A place p is k -bounded if $M(p) \leq k$ for all reachable markings. A place is unbounded if no finite k exists. In the automata context, unbounded state places are structurally impossible (exactly one token circulates within each automaton's state places). Unboundedness can only arise in channel places: if automaton A writes to p_σ on every tick and automaton B reads from p_σ only occasionally, tokens accumulate. An unbounded channel place indicates a producer-consumer rate mismatch that will eventually overflow the device's event queue.

9.4.3 P-Invariant Analysis

A *P-invariant* (place invariant) is a non-negative integer vector $\mathbf{x}$ such that $\mathbf{x}^\top \cdot \mathbf{W} = \mathbf{0}$, where $\mathbf{W}$ is the incidence matrix. For any reachable marking M : $\mathbf{x}^\top \cdot M = \mathbf{x}^\top \cdot M^0$ (conserved quantity). The lifting construction guarantees that each automaton $\mathcal{A}_i$ contributes a P-invariant $\mathbf{x}_i$ with $x_i(s) = 1$ for all $s \in S_i$ and zero elsewhere, reflecting the fact that exactly one token (the current state) circulates within each automaton. Verifying that these invariants are present in the composed net serves as a correctness check on the lifting.

9.4.4 Contention Analysis

Resources declared in black-box contracts (Section 5.6) are modeled as mutual-exclusion places initialized with a token count equal to the resource capacity (1 for an exclusive resource). Two automata $\mathcal{A}_i$ and $\mathcal{A}_j$ *contend* on resource r if there exists a reachable marking from which both can reach a state that requires r . The analyzer identifies all contending automaton pairs and reports the resource and the states involved, enabling the developer to add explicit arbitration logic (e.g., a third automaton acting as a token-passing mutex).

9.5 Showcase Scenarios

Two showcase categories directly exercise the Petri net analysis.

9.5.1 Showcase 13 — Petri Signal Chain

The signal chain consists of four automata arranged as a linear pipeline:

$$\textit{CommandRouter} \longrightarrow \textit{SafetyGate} \longrightarrow \textit{DriveUnit} \longrightarrow \textit{TelemetryObserver}$$

Each automaton reads from one channel place and writes to the next. After lifting and composition, the analyzer produces a four-subnet net with three channel places. P-invariant analysis confirms that one token circulates in each subnet. Reachability analysis reveals that the safety gate has a *locked* state from which it does not emit to the drive unit; under sustained command injection this state causes the channel place between gate and drive unit to grow unbounded, flagging a potential queue overflow. The showcase is designed so that this bottleneck is structurally visible from the Petri net without requiring fault injection.

9.5.2 Showcase 14 — Petri Contention

Three automata compete for a shared power budget modeled as a resource place with capacity 2 (two power units available):

- **ChargerNode** requests 1 unit when active,
- **MotionAxis** requests 1 unit when moving,
- **PowerAllocator** requests 1 unit during peak arbitration.

The analyzer identifies three contending pairs (charger+axis, charger+allocator, axis+allocator) and reports that the worst-case demand of 3 units exceeds the capacity of 2. The analysis output names the states from which simultaneous requests are possible, providing the developer with precise targets for adding priority or mutual-exclusion transitions to the power allocator automaton.

Chapter 10

Showcase Catalog

The system is validated by a catalog of 18 showcase automata organized into 14 categories. All definitions reside under `example/automata/showcase/`.

Table 10.1: Showcase automata catalog.

#	Category	Key concepts demonstrated
01	Basics	Timed and classic transitions, manual override input
02	Control	Event threshold triggers, pump state control
03	Probabilistic	Weighted 3-lane quality routing
04	Resilience	Watchdog timer, fault detection, retry with backoff
05	Energy	Load prediction, peak shaving scheduler
06	Pipeline	Multi-stage line buffer dispatcher
07	Folderized	Folder-based code layout, door safety controller
08	ESP32	PWM, LED, OLED, thermostat, production line (8 automata)
09	MCXN947	GPIO, buttons, touch, temperature on ARM Cortex-M7 (4 automata)
10	Guarded Cell	6-automata safety supervision network
11	Bidir. Loop	4 automata with bidirectional signal wiring
12	Black Box	Docker-sandboxed probe, external system wrapping
13	Petri Sig. Chain	Command router → safety gate → drive unit → telemetry
14	Petri Contention	Charger, motion axis, power allocator competing for resources

10.1 Selected Showcase: Quality Router (03)

The quality router demonstrates probabilistic transition selection. Three output lanes (A, B, C) are represented as transitions from a single routing state. Each transition carries a weight:

- Lane A: weight 5000 (50%)
- Lane B: weight 3000 (30%)
- Lane C: weight 2000 (20%)

On each tick the transition resolver applies Equation (5.3) to select the lane. The weights are static in this example but could be Lua expressions that respond to sensor readings (e.g., routing more batches to Lane A when Lane B is congested).

10.2 Selected Showcase: Sensor Watchdog Recovery (04)

The watchdog demonstrates timed transitions combined with classic guarded recovery. Figure 10.1 shows the full state machine.

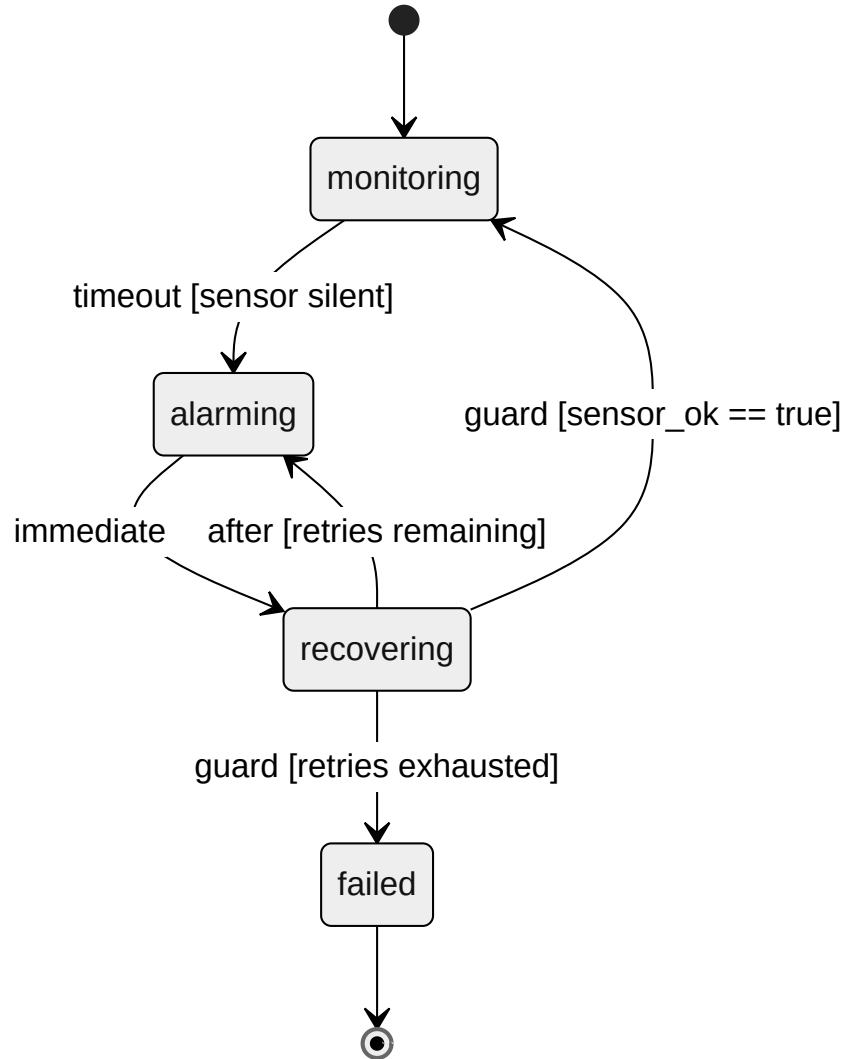


Figure 10.1: Sensor watchdog automaton. Dashed transitions are timed; solid transitions are classic guards. The **failed** state is a terminal escalation sink.

The automaton uses three transition types together: a *timed/timeout* transition detects sensor silence, *immediate* transitions chain states without observable intermediate steps, and *classic* guards check the `retry_count` variable to decide between another attempt and escalation to **failed**. This pattern recurs in many real-world IoT deployments: detect de-

vice failure, raise an alarm, attempt structured recovery, and escalate definitively if recovery is exhausted.

Chapter 11

End-to-End Demonstration

This chapter demonstrates Aetherium Automata as a running system. The walkthrough follows the complete operator workflow: start the infrastructure, open the IDE, load the demo project, deploy automata to a device, observe live behavior, inject a fault, replay execution history, and inspect the Petri net projection. All steps are executed against the reference Docker Compose stack that ships with the repository.

11.1 Infrastructure Setup

The reference deployment runs all backend services as Docker containers defined in `src/docker-compose.yaml`. The core development stack comprises three services:

gateway The Elixir/Phoenix gateway, exposed on `localhost:8080`. All IDE and device connections route through this service.

server3 An Aetherium server instance (`SERVER_ID=svr_03`) that manages automata life-cycle and owns the execution trace store for the showcase device.

device1 A C++17 engine process running in a container, pre-registered with **server3** and identified as `device_cpp_01`.

Starting the stack requires a single command from the `src/` directory:

```
1 make up # equivalent to: docker compose up -d gateway server3 device1
```

Listing 11.1: Starting the development stack.

The Makefile wraps `docker compose` for convenience and supports additional profiles for time-series monitoring (`make up-ts`) and black-box devices (`make up-blackbox`).

Once all containers report as healthy, the IDE can be launched:

```
1 cd src/ide
2 npm run dev # starts electron-vite dev server + Electron window
```

Listing 11.2: Launching the Electron IDE.

The IDE connects to the gateway at `ws://localhost:8080` using the operator token configured in the Docker environment (`GATEWAY_OPERATOR_TOKEN=dev_secret_token`). Connection status is displayed in the Gateway panel and the header status indicator.

11.2 Opening the Demo Project

The repository ships a ready-made demonstration project, `NewProject.aeth`, located in the repository root. The `.aeth` format is a JSON document that encodes all networks, devices, and their associated YAML automata paths, as described in Section 5.1.

Opening the file via *File* → *Open Project* loads all networks into the Network panel. Each network card shows its name, the number of devices it contains, and whether any of those devices are currently connected through the gateway. Figure 11.1 shows the Network panel with the demo project loaded and `device_cpp_01` online.



Figure 11.1: Network panel showing the demo project with one connected device.

The Explorer panel on the left lists the automata YAML files that belong to the project. Selecting a file opens it in the Monaco-based editor, where the YAML source can be inspected and edited directly. Changes are saved back to disk and can be re-deployed without restarting the stack.

11.3 Deploying Automata

Deploying an automaton to a device is an explicit operator action performed from the Runtime Monitor panel. The operator selects a target device from the device list, chooses one or more automata definitions, and clicks *Deploy*. The IDE sends a `deploy` message over the Phoenix channel, the server validates the YAML, compiles the EFSM model, and delivers the binary payload to the device engine. A deployment acknowledgement appears in the Runtime Monitor panel within a few hundred milliseconds on a local network.



Figure 11.2: Runtime Monitor panel after deploying the sensor watchdog automaton to `device_cpp_01`.

Once deployed, the automaton begins executing immediately. The engine sends periodic state snapshots — containing current state, all variable values, and the last fired transition — to the server, which forwards them to any subscribed IDE clients over the Phoenix channel.

11.4 Live Runtime Monitoring

The Runtime Monitor panel (Figure 11.3) provides a per-device, per-automaton view of live execution. For each deployed automaton it shows:

- the **current state** name, highlighted in the state list,
- **all variable values** with their types and current readings,
- **last transition** that fired, including its type and timestamp,
- a running **tick counter** showing how many evaluation cycles have completed.

The update rate is determined by the engine’s tick interval (configurable per deployment, default 100 ms) and the Phoenix channel push rate. Because the gateway maintains persistent WebSocket connections in both directions, state updates arrive in the IDE within one to two tick cycles of firing.



Figure 11.3: Runtime Monitor panel displaying live state and variable values for the deployed automaton.

The monitor is read-only but supports two direct actions available via the panel toolbar: *Start* and *Stop* send lifecycle commands to the device engine, transitioning the automaton between its active and idle lifecycle states without un-deploying it.

11.5 Injecting Faults

The fault injection subsystem is accessed from the Gateway panel. The operator selects a target device and chooses from six fault profiles:

delay Adds a configurable latency (milliseconds) to every outbound message from the device.

jitter Applies a random perturbation within a specified range to message delivery timing.

drop Drops a fraction of outbound messages, simulating packet loss.

disconnect Closes the device's WebSocket connection entirely for a specified duration, simulating a network outage.

battery_drain Injects a battery-level variable override to simulate low-power conditions.

none Clears all active fault profiles and restores normal operation.

Each profile accepts a *seed* value that initializes the random number generator used internally, making fault injection runs fully reproducible. The same seed applied to the same automaton always produces the same sequence of dropped or delayed messages, which is essential for debugging timing-dependent failures.

Figure 11.4 shows the Gateway panel with a **delay** fault profile active on `device_cpp_01`. The Runtime Monitor simultaneously shows the automaton continuing to execute normally

despite the injected latency, demonstrating that the engine’s internal timing is decoupled from transport delays.



Figure 11.4: Gateway panel with an active delay fault profile on the demo device.

11.6 Time-Travel Debugging

Every state transition and variable change forwarded from the device is persisted server-side in a per-deployment time series. Periodic full-state snapshots are interleaved with the event stream so that any past instant can be reconstructed without replaying the entire history from the beginning.

The time-travel workflow proceeds as follows:

1. The operator opens the Runtime Monitor panel. The panel shows the live execution timeline as a scrollable list of events: state transitions, variable deltas, and wall-clock timestamps.
2. The operator selects any event in the past and clicks *Rewind to Here*. The IDE sends a `rewind_deployment` WebSocket command to the gateway, carrying the target timestamp in milliseconds.
3. The server calls `TimeSeriesQuery.replay_state_at` to reconstruct the complete automaton state (active state name and all variable values) at the requested timestamp.
4. The server dispatches a `RestoreState` binary protocol message (0x48) to the physical device. The C++ engine pauses, applies the state snapshot atomically — setting `ctx_.currentState` and all variable values without firing any hooks — and waits in `Paused` state.
5. A `Resume` command is sent automatically, returning the device to `Running` from the restored historical state.

Figure 11.5 shows the replay timeline for the sensor watchdog automaton. The selected event shows the automaton entering the `alarming` state and the `alarm_count` variable incrementing.



Figure 11.5: Execution trace replay timeline in the Runtime Monitor panel.

The rewind is a true device-level operation, not a visual simulation: after `RestoreState` the device immediately fires the first transition consistent with the restored variable values, reproducing the same execution path that followed the selected event during the original run. This makes time-travel debugging effective even for timing-sensitive faults that cannot be reproduced by simply restarting the automaton.

11.7 Petri Net Analysis

The Petri Net panel takes the deployed network as input and applies the lifting procedure described in Section 9.2. Each automaton becomes a Petri net subnet; shared output/input channel pairs become the connecting arcs between subnets. The IDE renders the resulting net as an interactive graph.

Figure 11.6 shows the Petri net projection for the *Petri Contention* showcase (Showcase 14): three automata (charger node, motion axis, power allocator) share a single power-budget channel. The Petri net makes the contention explicit: the power-budget place has three input arcs and one output arc, which the structural analysis immediately flags as a potential bottleneck.



Figure 11.6: Petri Net panel showing token flow for the power-contention scenario.

The four analysis operations available through the panel are:

Reachability check Determines whether every automaton can eventually reach its terminal state (if any is defined) from the initial marking.

Deadlock detection Identifies markings from which no transition is enabled, indicating that the network could reach a permanently stuck configuration.

Bottleneck detection Flags places with a high ratio of input arcs to output arcs, which are structural candidates for throughput-limiting resources.

Contention report Lists pairs of automata that compete for the same resource place, providing the operator with the information needed to add arbitration logic.

The analysis results are displayed inline in the panel as a structured report beneath the net graph.

11.8 Analyzer Panel

The Analyzer panel complements the Petri net view with a higher-level projection over the live deployment. It queries the server for the current analyzer report, which the server computes from the registered automata metadata and recent transition history.

The report contains three sections:

- **Device summary** — for each connected device: uptime, number of deployed automata, tick rate, and number of transitions fired since deployment.
- **Channel activity** — for each named output/input pair: message count, last value, and whether the channel is currently saturated (output producing faster than input is consuming).

- **Anomaly log** — entries flagged by the server’s anomaly detector, such as a device going silent, a variable exceeding a configured threshold, or an automaton spending an unusually long time in a single state.

Figure 11.7 shows the Analyzer panel during a deployment of the Petri Signal Chain showcase (Showcase 13), where the safety gate automaton is identified as processing commands at a lower rate than the command router is emitting them.



Figure 11.7: Analyzer panel reporting channel activity and an anomaly for the signal chain scenario.

11.9 Black-Box Devices

The black-box feature allows external systems that cannot be instrumented with the Aetherium engine to participate in the same orchestration layer. A black-box device exposes only a named signal contract; its internal implementation is opaque to the IDE.

The reference deployment includes a black-box container (`blackbox1`) that wraps a C++ engine running the Docker black-box probe automaton from showcase 12. Starting this stack variant:

```
1 make up-blackbox # gateway + server3 + blackbox1
2 make smoke-blackbox
```

Listing 11.3: Starting the black-box demonstration stack.

The smoke test deploys the probe automaton, sends a sequence of activation messages, and verifies that the expected output messages arrive within the configured timeout. All interactions are visible in the Runtime Monitor panel under the `black_box_01` device entry, but the panel only shows the contract (inputs and outputs) — not the internal state of the automaton, in keeping with the black-box semantics.

11.10 Demonstration Summary

The end-to-end walkthrough demonstrates all primary platform capabilities on a single machine without any physical hardware. Table 11.1 maps each demonstrated capability to the platform component responsible for it.

Table 11.1: Demonstration coverage summary.

Capability	Platform component	Demo section
Stack startup	Docker Compose / Makefile	§11.1
Project file open	IDE file handler	§11.2
Automaton deployment	Server + engine deploy API	§11.3
Live state monitoring	Runtime Monitor panel	§11.4
Fault injection	Gateway fault service	§11.5
Time-travel debugging	Execution trace + replay	§11.6
Petri net analysis	Petri Net panel + lifting	§11.7
Anomaly reporting	Analyzer panel	§11.8
Black-box contract	Black-box device type	§11.9

The hardware-in-the-loop variants (ESP32, MCXN947, Raspberry Pi Pico) follow the same IDE workflow; the only difference is that **make up-gateway** replaces the full Docker stack and the server runs on the host machine with a serial transport connector. This confirms that the platform’s layered architecture — described in Chapter 4 — successfully isolates the operator-facing workflow from the physical transport.

Chapter 12

Conclusion

12.1 Summary

This thesis presented Aetherium Automata, a platform for the visual design, deployment, and monitoring of distributed IoT control systems modeled as Extended Finite State Machines. The system was designed around two interrelated principles introduced in Section 1.1.

Reality-in-the-Loop holds that production code and testing infrastructure must share the same execution substrate. Rather than maintaining a separate test harness, Aetherium lets developers run the actual automaton on actual (or emulated) hardware while the fault injection layer perturbs the communication boundary at configurable rates. Probabilistic transitions make rare real-world events reliably inducible; black-box contracts define the formal interface under test; and time-travel debugging closes the loop by allowing the developer to rewind to the exact state sequence that preceded any observed anomaly. Events that would otherwise require weeks of field observation can be forced, observed, and diagnosed in a single development session.

Deployment-Aware Development holds that the developer should never be blind to the network their automata run on. Because Aetherium owns the communication layer end-to-end, every deployed device is accompanied by live telemetry — RTT, connection type, battery level, wake-up timer intervals — available in the same IDE panel as the state machine. Developers can make quantified, deliberate trade-offs instead of guessing at network conditions. The Petri net analysis layer extends this to the structural level: lifting all automata in the network to a single Petri net makes bottlenecks, deadlocks, and resource contention computable and visualizable, giving the developer a vantage point that is inaccessible when inspecting each device in isolation.

The concrete contributions that realise these principles are:

1. A formal EFSM model supporting five transition types (classic, timed, event, probabilistic, immediate), typed variables with reactive change detection, and Lua-scripted guards and actions. The probabilistic type is the engine of RitL.
2. A YAML domain-specific language for authoring automata in both inline and folder layouts, with a parser that validates structure and identifier correctness.
3. A portable C++17 runtime engine that executes the model on desktop Linux and embedded platforms (ESP32, Raspberry Pi Pico, MCXN947) via three abstract platform interfaces (IClock, IRandomSource, IScriptEngine).

4. A binary wire protocol inspired by MQTT’s lightweight publish/subscribe model but reimplemented to support server-side sequence numbering and execution trace recording — the technical foundation of time-travel debugging.
5. An Electron IDE providing live state monitoring, variable inspection, fault injection, time-travel debugging, and Petri net analysis — the single surface at which both RitL and DAD are exercised.
6. A Petri net analysis layer that lifts multi-automata networks to a structural plane where deadlocks, bottlenecks, and resource contention are computed rather than inferred — the highest expression of Deployment-Aware Development.
7. A catalog of 18 showcase automata covering real-world IoT scenarios including resilience, energy management, probabilistic routing, and multi-device coordination.

12.2 Future Work

The following directions are identified for future development:

- Full Petri net reachability analysis (currently the analyzer performs structural checks; complete reachability graph construction is pending).
- Bytecode compilation of Lua scripts into a compact `EngineBytecodeProgram` format for deployment to the most constrained targets.
- Hierarchical and abstract automata — template (parameterized) automata instantiated at compose time, with Petri net lifting extended to hierarchical networks, moving the system toward a full DEVS-style coupled-model composition.
- Third-party device interoperability via black-box adapters: the black-box contract architecture (Section 5.6) makes it straightforward to wrap external subsystems — ROS2 nodes, Zigbee gateways, WiFi relays — with a thin protocol adapter that exposes only the signal contract, without modifying the core engine or gateway.
- Extended IDE canvas: undo/redo history, multi-select, copy-paste of states and transitions.
- Integration with InfluxDB and Grafana for long-running time-series metrics dashboards.

Bibliography

- [1] ARMSTRONG, J. *Programming Erlang: Software for a Concurrent World*. Raleigh, NC: Pragmatic Bookshelf, 2007. ISBN 978-1-934356-00-5.
- [2] BEILHARZ, J.; WIESNER, P.; BOOCKMEYER, A.; FRIEDENBERGER, D.; BROKHAUSEN, F. et al. Continuously Testing Distributed IoT Systems: An Overview of the State of the Art. In: *Service-Oriented Computing – ICSOC 2021 Workshops*. Cham: Springer, 2022, p. 375–387.
- [3] BIGA, J. P. M. *RapidYAML: Rapid YAML — a library to parse and emit YAML, and do it fast* online. 2024. Available at: <https://github.com/biojppm/rapidyaml>. [cit. 2025-05-11]. MIT License.
- [4] ECLIPSE FOUNDATION. *Eclipse 4diac: Open Source Infrastructure for Distributed Industrial Control Based on IEC 61499* online. 2024. Available at: <https://eclipse.dev/4diac/>. [cit. 2025-05-11].
- [5] ENGBLOM, J. A Review of Reverse Debugging. In: *2012 System, Software, SoC and Silicon Debug Conference (S4D)*. IEEE, 2012, p. 1–6.
- [6] GEELS, D.; ALTEKAR, G.; SHENKER, S. and STOICA, I. Replay Debugging for Distributed Applications. In: *2006 USENIX Annual Technical Conference (USENIX ATC 06)*. Boston, MA: USENIX Association, 2006, p. 289–300. Available at: <https://www.usenix.org/conference/2006-usenix-annual-technical-conference/replay-debugging-distributed-applications>. Best Paper Award.
- [7] HAREL, D. Statecharts: A Visual Formalism for Complex Systems. *Science of Computer Programming*. North-Holland, 1987, vol. 8, no. 3, p. 231–274. ISSN 0167-6423.
- [8] HOPCROFT, J. E.; MOTWANI, R. and ULLMAN, J. D. *Introduction to Automata Theory, Languages, and Computation*. 3rd ed. Upper Saddle River, NJ: Pearson Education, 2006. ISBN 978-0-321-45536-9.
- [9] IERUSALIMSCHY, R.; DE FIGUEIREDO, L. H. and CELES, W. *Lua 5.4 Reference Manual* online. Lua.org, PUC-Rio, 2020. Available at: <https://www.lua.org/manual/5.4/>. [cit. 2025-05-11].
- [10] IERUSALIMSCHY, R.; DE FIGUEIREDO, L. H. and CELES FILHO, W. Lua—An Extensible Extension Language. *Software: Practice and Experience*. Wiley, 1996, vol. 26, no. 6, p. 635–652. ISSN 1097-024X. Available at: [https://doi.org/10.1002/\(SICI\)1097-024X\(199606\)26:6<635::AID-SPE26>3.0.CO;2-P](https://doi.org/10.1002/(SICI)1097-024X(199606)26:6<635::AID-SPE26>3.0.CO;2-P).

- [11] LEBLANC, T. J. and MELLOR CRUMMEY, J. M. Debugging Parallel Programs with Instant Replay. *IEEE Transactions on Computers*. IEEE, 1987, vol. 36, no. 4, p. 471–482. ISSN 0018-9340.
- [12] MACHINE ZONE, INC.. *IXWebSocket: WebSocket and HTTP Client and Server Library* online. 2024. Available at: <https://github.com/machinezone/IXWebSocket>. [cit. 2025-05-11]. BSD 3-Clause License.
- [13] META OPEN SOURCE. *React: The Library for Web and Native User Interfaces* online. 2024. Available at: <https://react.dev/>. [cit. 2025-05-11].
- [14] MURATA, T. Petri Nets: Properties, Analysis and Applications. *Proceedings of the IEEE*. IEEE, 1989, vol. 77, no. 4, p. 541–580. ISSN 0018-9219.
- [15] NOAMAN, M.; KHAN, M. S.; ABRAR, M. F.; ALI, S.; ALVI, A. et al. Challenges in Integration of Heterogeneous Internet of Things. *Scientific Programming*. Hindawi, 2022, vol. 2022, p. 1–17.
- [16] O’LEARY, N. and CONWAY-JONES, D. *Node-RED: Low-Code Programming for Event-Driven Applications* online. 2013. Available at: <https://nodered.org/>. [cit. 2025-05-11]. Open-sourced September 2013; now part of the OpenJS Foundation.
- [17] OPENJS FOUNDATION. *Electron: Build Cross-Platform Desktop Apps with JavaScript, HTML, and CSS* online. 2024. Available at: <https://www.electronjs.org/>. [cit. 2025-05-11].
- [18] PETRI, C. A. *Kommunikation mit Automaten*. Darmstadt, Germany, 1962. Dissertation. Technische Hochschule Darmstadt.
- [19] PHOENIX FRAMEWORK CONTRIBUTORS. *Phoenix Framework — Peace of Mind from Prototype to Production* online. 2024. Available at: <https://www.phoenixframework.org/>. [cit. 2025-05-11].
- [20] POIMANDRES (PMNDRS). *Zustand: Bear Necessities for State Management in React* online. 2024. Available at: <https://github.com/pmndrs/zustand>. [cit. 2025-05-11]. MIT License.
- [21] STRASSER, T.; ROOKER, M. N.; EBENHOFER, G.; ZOITL, A.; SUNDER, C. et al. Framework for Distributed Industrial Automation and Control (4DIAC). In: *2008 6th IEEE International Conference on Industrial Informatics (INDIN)*. IEEE, 2008, p. 283–288.
- [22] THEPHD. *Sol2: C++ ↔ Lua API Wrapper* online. 2023. Available at: <https://github.com/ThePhD/sol2>. [cit. 2025-05-11]. MIT License.
- [23] VALIM, J. et al. *Elixir Programming Language* online. 2024. Available at: <https://elixir-lang.org/>. [cit. 2025-05-11].
- [24] YAML LANGUAGE DEVELOPMENT TEAM. *YAML Ain’t Markup Language (YAML) Revision 1.2.2* online. 2021. Available at: <https://yaml.org/spec/1.2.2/>. [cit. 2025-05-11].

- [25] ZOITL, A. and STRASSER, T. *Distributed Control Applications: Guidelines, Design Patterns, and Application Examples with the IEC 61499*. Boca Raton, FL: CRC Press / Taylor & Francis, 2017. ISBN 978-1-482-25905-6.

Appendix A

Contents of the Included Storage Media

The included storage medium contains the following items:

src/ Full source code of the Aetherium Automata platform:

- **src/engine/** — C++17 runtime engine and platform targets
- **src/ide/** — Electron IDE (TypeScript/React)
- **src/gateway/** — Elixir/Phoenix gateway
- **src/server/** — Elixir server

example/ Showcase automata catalog:

- **example/automata/showcase/** — 18 YAML automata definitions
- **example/ide_demo_projects/** — IDE demo project files (**.aeth**)

thesis/ $\text{\LaTeX}$ source of this technical report and compiled PDF.

Appendix B

YAML DSL Schema Reference

The complete field reference for the YAML domain-specific language is given below. Fields marked **(R)** are required; fields marked **(O)** are optional.

Table B.1: Top-level YAML fields.

Field	Req.	Description
<code>version</code>	R	Schema version (currently <code>0.0.1</code>)
<code>config</code>	R	Automaton metadata block
<code>variables</code>	O	List of variable definitions
<code>automata</code>	R	EFSM body

Table B.2: `config` block fields.

Field	Req.	Description
<code>name</code>	R	Human-readable automaton name
<code>type</code>	R	<code>inline</code> or <code>folder</code>
<code>location</code>	R if folder	Path to the folder containing code files
<code>language</code>	O	Scripting language; default <code>lua</code>
<code>description</code>	O	Free-text description
<code>author</code>	O	Author name
<code>tags</code>	O	List of string tags for discovery

Table B.3: Transition type-specific fields.

Type	Required fields	Optional fields
<code>classic</code>	<code>condition</code>	<code>priority</code> , <code>weight</code>
<code>timed</code>	<code>timed.mode</code> ; timing field depends on mode (<code>after</code> , <code>every</code> , ...)	<code>timed.jitter</code> , <code>priority</code>
<code>event</code>	<code>event.triggers</code>	<code>event.require_all</code> , <code>priority</code>
<code>probabilistic</code>	<code>weight</code>	<code>priority</code>
<code>immediate</code>	—	<code>priority</code>

Appendix C

Supported Platform Targets

Table C.1: Engine platform targets and their capability profiles.

Target		Architecture		Script engine		Transport	Build	
Desktop		x86-64	/	Lua 5.4 (Sol2)		WebSocket	CMake	
Linux/macOS		ARM64				(IXWebSocket)		
ESP32		Xtensa LX6		Lightweight	/	Serial / MQTT	Arduino	+
				no-op			CMake	
Raspberry Pi		ARM	Cortex-	No-op stub		Serial (UART)	CMake	
Pico		M0+						
MCXN947		ARM	Cortex-	Lightweight		Serial (UART)	CMake	
		M7						